

libgpod
2.2.4

Generated by Doxygen 1.9.8

1 Topic Index	1
1.1 Topics	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Topic Documentation	7
4.1 GPIO chips	7
4.1.1 Detailed Description	7
4.1.2 Function Documentation	8
4.1.2.1 gpiod_chip_close()	8
4.1.2.2 gpiod_chip_get_fd()	8
4.1.2.3 gpiod_chip_get_info()	8
4.1.2.4 gpiod_chip_get_line_info()	9
4.1.2.5 gpiod_chip_get_line_offset_from_name()	9
4.1.2.6 gpiod_chip_get_path()	9
4.1.2.7 gpiod_chip_open()	10
4.1.2.8 gpiod_chip_read_info_event()	10
4.1.2.9 gpiod_chip_request_lines()	11
4.1.2.10 gpiod_chip_unwatch_line_info()	11
4.1.2.11 gpiod_chip_wait_info_event()	11
4.1.2.12 gpiod_chip_watch_line_info()	12
4.2 Chip info	12
4.2.1 Detailed Description	12
4.2.2 Function Documentation	13
4.2.2.1 gpiod_chip_info_free()	13
4.2.2.2 gpiod_chip_info_get_label()	13
4.2.2.3 gpiod_chip_info_get_name()	13
4.2.2.4 gpiod_chip_info_get_num_lines()	13
4.3 Line definitions	14
4.3.1 Detailed Description	14
4.3.2 Enumeration Type Documentation	14
4.3.2.1 gpiod_line_bias	14
4.3.2.2 gpiod_line_clock	15
4.3.2.3 gpiod_line_direction	15
4.3.2.4 gpiod_line_drive	15
4.3.2.5 gpiod_line_edge	16
4.3.2.6 gpiod_line_value	16
4.4 Line info	16
4.4.1 Detailed Description	17

4.4.2 Function Documentation	17
4.4.2.1 <code>gpiod_line_info_copy()</code>	17
4.4.2.2 <code>gpiod_line_info_free()</code>	17
4.4.2.3 <code>gpiod_line_info_get_bias()</code>	18
4.4.2.4 <code>gpiod_line_info_get_consumer()</code>	18
4.4.2.5 <code>gpiod_line_info_get_debounce_period_us()</code>	18
4.4.2.6 <code>gpiod_line_info_get_direction()</code>	19
4.4.2.7 <code>gpiod_line_info_get_drive()</code>	19
4.4.2.8 <code>gpiod_line_info_get_edge_detection()</code>	19
4.4.2.9 <code>gpiod_line_info_get_event_clock()</code>	20
4.4.2.10 <code>gpiod_line_info_get_name()</code>	20
4.4.2.11 <code>gpiod_line_info_get_offset()</code>	20
4.4.2.12 <code>gpiod_line_info_is_active_low()</code>	21
4.4.2.13 <code>gpiod_line_info_is_debounced()</code>	21
4.4.2.14 <code>gpiod_line_info_is_used()</code>	21
4.5 Line status watch events	22
4.5.1 Detailed Description	22
4.5.2 Enumeration Type Documentation	22
4.5.2.1 <code>gpiod_info_event_type</code>	22
4.5.3 Function Documentation	23
4.5.3.1 <code>gpiod_info_event_free()</code>	23
4.5.3.2 <code>gpiod_info_event_get_event_type()</code>	23
4.5.3.3 <code>gpiod_info_event_get_line_info()</code>	23
4.5.3.4 <code>gpiod_info_event_get_timestamp_ns()</code>	24
4.6 Line settings objects	24
4.6.1 Detailed Description	25
4.6.2 Function Documentation	25
4.6.2.1 <code>gpiod_line_settings_copy()</code>	25
4.6.2.2 <code>gpiod_line_settings_free()</code>	25
4.6.2.3 <code>gpiod_line_settings_get_active_low()</code>	26
4.6.2.4 <code>gpiod_line_settings_get_bias()</code>	26
4.6.2.5 <code>gpiod_line_settings_get_debounce_period_us()</code>	26
4.6.2.6 <code>gpiod_line_settings_get_direction()</code>	27
4.6.2.7 <code>gpiod_line_settings_get_drive()</code>	27
4.6.2.8 <code>gpiod_line_settings_get_edge_detection()</code>	27
4.6.2.9 <code>gpiod_line_settings_get_event_clock()</code>	28
4.6.2.10 <code>gpiod_line_settings_get_output_value()</code>	28
4.6.2.11 <code>gpiod_line_settings_new()</code>	28
4.6.2.12 <code>gpiod_line_settings_reset()</code>	29
4.6.2.13 <code>gpiod_line_settings_set_active_low()</code>	30
4.6.2.14 <code>gpiod_line_settings_set_bias()</code>	30
4.6.2.15 <code>gpiod_line_settings_set_debounce_period_us()</code>	30

4.6.2.16	<code>gpiod_line_settings_set_direction()</code>	31
4.6.2.17	<code>gpiod_line_settings_set_drive()</code>	31
4.6.2.18	<code>gpiod_line_settings_set_edge_detection()</code>	31
4.6.2.19	<code>gpiod_line_settings_set_event_clock()</code>	32
4.6.2.20	<code>gpiod_line_settings_set_output_value()</code>	32
4.7	Line configuration objects	32
4.7.1	Detailed Description	33
4.7.2	Function Documentation	33
4.7.2.1	<code>gpiod_line_config_add_line_settings()</code>	33
4.7.2.2	<code>gpiod_line_config_free()</code>	34
4.7.2.3	<code>gpiod_line_config_get_configured_offsets()</code>	34
4.7.2.4	<code>gpiod_line_config_get_line_settings()</code>	34
4.7.2.5	<code>gpiod_line_config_get_num_configured_offsets()</code>	35
4.7.2.6	<code>gpiod_line_config_new()</code>	35
4.7.2.7	<code>gpiod_line_config_reset()</code>	35
4.7.2.8	<code>gpiod_line_config_set_output_values()</code>	35
4.8	Request configuration objects	36
4.8.1	Detailed Description	36
4.8.2	Function Documentation	37
4.8.2.1	<code>gpiod_request_config_free()</code>	37
4.8.2.2	<code>gpiod_request_config_get_consumer()</code>	37
4.8.2.3	<code>gpiod_request_config_get_event_buffer_size()</code>	37
4.8.2.4	<code>gpiod_request_config_new()</code>	37
4.8.2.5	<code>gpiod_request_config_set_consumer()</code>	38
4.8.2.6	<code>gpiod_request_config_set_event_buffer_size()</code>	38
4.9	Line request operations	38
4.9.1	Detailed Description	39
4.9.2	Function Documentation	39
4.9.2.1	<code>gpiod_line_request_get_chip_name()</code>	39
4.9.2.2	<code>gpiod_line_request_get_fd()</code>	40
4.9.2.3	<code>gpiod_line_request_get_num_requested_lines()</code>	40
4.9.2.4	<code>gpiod_line_request_get_requested_offsets()</code>	40
4.9.2.5	<code>gpiod_line_request_get_value()</code>	41
4.9.2.6	<code>gpiod_line_request_get_values()</code>	41
4.9.2.7	<code>gpiod_line_request_get_values_subset()</code>	41
4.9.2.8	<code>gpiod_line_request_read_edge_events()</code>	42
4.9.2.9	<code>gpiod_line_request_reconfigure_lines()</code>	42
4.9.2.10	<code>gpiod_line_request_release()</code>	43
4.9.2.11	<code>gpiod_line_request_set_value()</code>	43
4.9.2.12	<code>gpiod_line_request_set_values()</code>	44
4.9.2.13	<code>gpiod_line_request_set_values_subset()</code>	44
4.9.2.14	<code>gpiod_line_request_wait_edge_events()</code>	44

4.10 Line edge events handling	45
4.10.1 Detailed Description	46
4.10.2 Enumeration Type Documentation	46
4.10.2.1 <code>gpiod_edge_event_type</code>	46
4.10.3 Function Documentation	46
4.10.3.1 <code>gpiod_edge_event_buffer_free()</code>	46
4.10.3.2 <code>gpiod_edge_event_buffer_get_capacity()</code>	46
4.10.3.3 <code>gpiod_edge_event_buffer_get_event()</code>	47
4.10.3.4 <code>gpiod_edge_event_buffer_get_num_events()</code>	47
4.10.3.5 <code>gpiod_edge_event_buffer_new()</code>	47
4.10.3.6 <code>gpiod_edge_event_copy()</code>	48
4.10.3.7 <code>gpiod_edge_event_free()</code>	48
4.10.3.8 <code>gpiod_edge_event_get_event_type()</code>	48
4.10.3.9 <code>gpiod_edge_event_get_global_seqno()</code>	49
4.10.3.10 <code>gpiod_edge_event_get_line_offset()</code>	49
4.10.3.11 <code>gpiod_edge_event_get_line_seqno()</code>	49
4.10.3.12 <code>gpiod_edge_event_get_timestamp_ns()</code>	50
4.11 Stuff that didn't fit anywhere else	50
4.11.1 Detailed Description	50
4.11.2 Function Documentation	51
4.11.2.1 <code>gpiod_api_version()</code>	51
4.11.2.2 <code>gpiod_is_gpiochip_device()</code>	51
5 Class Documentation	53
5.1 <code>gpiod_chip</code> Struct Reference	53
5.1.1 Detailed Description	53
5.2 <code>gpiod_chip_info</code> Struct Reference	53
5.2.1 Detailed Description	54
5.3 <code>gpiod_edge_event</code> Struct Reference	54
5.3.1 Detailed Description	54
5.4 <code>gpiod_edge_event_buffer</code> Struct Reference	54
5.4.1 Detailed Description	55
5.5 <code>gpiod_info_event</code> Struct Reference	55
5.5.1 Detailed Description	55
5.6 <code>gpiod_line_config</code> Struct Reference	56
5.6.1 Detailed Description	56
5.7 <code>gpiod_line_info</code> Struct Reference	56
5.7.1 Detailed Description	57
5.8 <code>gpiod_line_request</code> Struct Reference	57
5.8.1 Detailed Description	57
5.9 <code>gpiod_line_settings</code> Struct Reference	57
5.9.1 Detailed Description	58

5.10 gpod_request_config Struct Reference	58
5.10.1 Detailed Description	58
5.11 per_line_config Struct Reference	59
5.12 settings_node Struct Reference	59
6 File Documentation	61
6.1 src/include/gpod.h File Reference	61
6.2 gpod.h	66
6.3 internal.h	71
Index	73

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

GPIO chips	7
Chip info	12
Line definitions	14
Line info	16
Line status watch events	22
Line settings objects	24
Line configuration objects	32
Request configuration objects	36
Line request operations	38
Line edge events handling	45
Stuff that didn't fit anywhere else	50

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

gpiod_chip	53
gpiod_chip_info	53
gpiod_edge_event	54
gpiod_edge_event_buffer	54
gpiod_info_event	55
gpiod_line_config	56
gpiod_line_info	56
gpiod_line_request	57
gpiod_line_settings	57
gpiod_request_config	58
per_line_config	59
settings_node	59

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/include/ gpiod.h	61
src/lib/ internal.h	71

Chapter 4

Topic Documentation

4.1 GPIO chips

Functions

- struct `gpiod_chip` * `gpiod_chip_open` (const char *path)
Open a chip by path.
- void `gpiod_chip_close` (struct `gpiod_chip` *chip)
Close the chip and release all associated resources.
- struct `gpiod_chip_info` * `gpiod_chip_get_info` (struct `gpiod_chip` *chip)
Get information about the chip.
- const char * `gpiod_chip_get_path` (struct `gpiod_chip` *chip)
Get the path used to open the chip.
- struct `gpiod_line_info` * `gpiod_chip_get_line_info` (struct `gpiod_chip` *chip, unsigned int offset)
Get a snapshot of information about a line.
- struct `gpiod_line_info` * `gpiod_chip_watch_line_info` (struct `gpiod_chip` *chip, unsigned int offset)
Get a snapshot of the status of a line and start watching it for future changes.
- int `gpiod_chip_unwatch_line_info` (struct `gpiod_chip` *chip, unsigned int offset)
Stop watching a line for status changes.
- int `gpiod_chip_get_fd` (struct `gpiod_chip` *chip)
Get the file descriptor associated with the chip.
- int `gpiod_chip_wait_info_event` (struct `gpiod_chip` *chip, int64_t timeout_ns)
Wait for line status change events on any of the watched lines on the chip.
- struct `gpiod_info_event` * `gpiod_chip_read_info_event` (struct `gpiod_chip` *chip)
Read a single line status change event from the chip.
- int `gpiod_chip_get_line_offset_from_name` (struct `gpiod_chip` *chip, const char *name)
Map a line's name to its offset within the chip.
- struct `gpiod_line_request` * `gpiod_chip_request_lines` (struct `gpiod_chip` *chip, struct `gpiod_request_config` *req_cfg, struct `gpiod_line_config` *line_cfg)
Request a set of lines for exclusive usage.

4.1.1 Detailed Description

Functions and data structures for GPIO chip operations.

A GPIO chip object is associated with an open file descriptor to the GPIO character device. It exposes basic information about the chip and allows callers to retrieve information about each line, watch lines for state changes and make line requests.

4.1.2 Function Documentation

4.1.2.1 `gpiod_chip_close()`

```
void gpiod_chip_close (
    struct gpiod\_chip * chip )
```

Close the chip and release all associated resources.

Parameters

<i>chip</i>	Chip to close.
-------------	----------------

4.1.2.2 `gpiod_chip_get_fd()`

```
int gpiod_chip_get_fd (
    struct gpiod\_chip * chip )
```

Get the file descriptor associated with the chip.

Parameters

<i>chip</i>	GPIO chip object.
-------------	-------------------

Returns

File descriptor number for the chip.

This function never fails. The returned file descriptor must not be closed by the caller. Call [gpiod_chip_close](#) to close the file descriptor by closing the chip owning it.

4.1.2.3 `gpiod_chip_get_info()`

```
struct gpiod\_chip\_info * gpiod_chip_get_info (
    struct gpiod\_chip * chip )
```

Get information about the chip.

Parameters

<i>chip</i>	GPIO chip object.
-------------	-------------------

Returns

New GPIO chip info object or NULL if an error occurred. The returned object must be freed by the caller using [gpiod_chip_info_free](#).

4.1.2.4 `gpiod_chip_get_line_info()`

```
struct gpiod\_line\_info * gpiod_chip_get_line_info (
    struct gpiod\_chip * chip,
    unsigned int offset )
```

Get a snapshot of information about a line.

Parameters

<i>chip</i>	GPIO chip object.
<i>offset</i>	The offset of the GPIO line.

Returns

New GPIO line info object or NULL if an error occurred. The returned object must be freed by the caller using [gpiod_line_info_free](#).

4.1.2.5 `gpiod_chip_get_line_offset_from_name()`

```
int gpiod_chip_get_line_offset_from_name (
    struct gpiod\_chip * chip,
    const char * name )
```

Map a line's name to its offset within the chip.

Parameters

<i>chip</i>	GPIO chip object.
<i>name</i>	Name of the GPIO line to map.

Returns

Offset of the line within the chip or -1 on error.

Note

If a line with given name is not exposed by the chip, the function sets `errno` to `ENOENT`.

4.1.2.6 `gpiod_chip_get_path()`

```
const char * gpiod_chip_get_path (
    struct gpiod\_chip * chip )
```

Get the path used to open the chip.

Parameters

<i>chip</i>	GPIO chip object.
-------------	-------------------

Returns

Path to the file passed as argument to [gpiod_chip_open](#). The returned pointer is valid for the lifetime of the chip object and must not be freed by the caller.

4.1.2.7 gpiod_chip_open()

```
struct gpiod\_chip * gpiod_chip_open (
    const char * path )
```

Open a chip by path.

Parameters

<i>path</i>	Path to the gpiochip device file.
-------------	-----------------------------------

Returns

GPIO chip object or NULL if an error occurred. The returned object must be closed by the caller using [gpiod_chip_close](#).

4.1.2.8 gpiod_chip_read_info_event()

```
struct gpiod\_info\_event * gpiod_chip_read_info_event (
    struct gpiod\_chip * chip )
```

Read a single line status change event from the chip.

Parameters

<i>chip</i>	GPIO chip object.
-------------	-------------------

Returns

Newly read watch event object or NULL on error. The event must be freed by the caller using [gpiod_info_event_free](#).

Note

If no events are pending, this function will block.

4.1.2.9 `gpiod_chip_request_lines()`

```
struct gpiod_line_request * gpiod_chip_request_lines (
    struct gpiod_chip * chip,
    struct gpiod_request_config * req_cfg,
    struct gpiod_line_config * line_cfg )
```

Request a set of lines for exclusive usage.

Parameters

<i>chip</i>	GPIO chip object.
<i>req_cfg</i>	Request config object. Can be NULL for default settings.
<i>line_cfg</i>	Line config object.

Returns

New line request object or NULL if an error occurred. The request must be released by the caller using [`gpiod\_line\_request\_release`](#).

4.1.2.10 `gpiod_chip_unwatch_line_info()`

```
int gpiod_chip_unwatch_line_info (
    struct gpiod_chip * chip,
    unsigned int offset )
```

Stop watching a line for status changes.

Parameters

<i>chip</i>	GPIO chip object.
<i>offset</i>	The offset of the line to stop watching.

Returns

0 on success, -1 on failure.

4.1.2.11 `gpiod_chip_wait_info_event()`

```
int gpiod_chip_wait_info_event (
    struct gpiod_chip * chip,
    int64_t timeout_ns )
```

Wait for line status change events on any of the watched lines on the chip.

Parameters

<i>chip</i>	GPIO chip object.
<i>timeout_ns</i>	Wait time limit in nanoseconds. If set to 0, the function returns immediately. If set to a negative number, the function blocks indefinitely until an event becomes available.

Returns

0 if wait timed out, -1 if an error occurred, 1 if an event is pending.

4.1.2.12 `gpiod_chip_watch_line_info()`

```
struct gpiod\_line\_info * gpiod_chip_watch_line_info (
    struct gpiod\_chip * chip,
    unsigned int offset )
```

Get a snapshot of the status of a line and start watching it for future changes.

Parameters

<i>chip</i>	GPIO chip object.
<i>offset</i>	The offset of the GPIO line.

Returns

New GPIO line info object or NULL if an error occurred. The returned object must be freed by the caller using [gpiod_line_info_free](#).

Note

Line status does not include the line value. To monitor the line value the line must be requested as an input with edge detection set.

4.2 Chip info**Functions**

- void [gpiod_chip_info_free](#) (struct [gpiod_chip_info](#) *info)
Free a chip info object and release all associated resources.
- const char * [gpiod_chip_info_get_name](#) (struct [gpiod_chip_info](#) *info)
Get the name of the chip as represented in the kernel.
- const char * [gpiod_chip_info_get_label](#) (struct [gpiod_chip_info](#) *info)
Get the label of the chip as represented in the kernel.
- size_t [gpiod_chip_info_get_num_lines](#) (struct [gpiod_chip_info](#) *info)
Get the number of lines exposed by the chip.

4.2.1 Detailed Description

Functions for retrieving kernel information about chips.

Line info object contains an immutable snapshot of a chip's status.

The chip info contains all the publicly available information about a chip.

Some accessor methods return pointers. Those pointers refer to internal fields. The lifetimes of those fields are tied to the lifetime of the containing chip info object. Such pointers remain valid until [gpiod_chip_info_free](#) is called on the containing chip info object. They must not be freed by the caller.

4.2.2 Function Documentation

4.2.2.1 `gpiod_chip_info_free()`

```
void gpiod_chip_info_free (
    struct gpiod_chip_info * info )
```

Free a chip info object and release all associated resources.

Parameters

<i>info</i>	GPIO chip info object to free.
-------------	--------------------------------

4.2.2.2 `gpiod_chip_info_get_label()`

```
const char * gpiod_chip_info_get_label (
    struct gpiod_chip_info * info )
```

Get the label of the chip as represented in the kernel.

Parameters

<i>info</i>	GPIO chip info object.
-------------	------------------------

Returns

Valid pointer to a human-readable string containing the chip label. The string lifetime is tied to the chip info object so the pointer must not be freed by the caller.

4.2.2.3 `gpiod_chip_info_get_name()`

```
const char * gpiod_chip_info_get_name (
    struct gpiod_chip_info * info )
```

Get the name of the chip as represented in the kernel.

Parameters

<i>info</i>	GPIO chip info object.
-------------	------------------------

Returns

Valid pointer to a human-readable string containing the chip name. The string lifetime is tied to the chip info object so the pointer must not be freed by the caller.

4.2.2.4 `gpiod_chip_info_get_num_lines()`

```
size_t gpiod_chip_info_get_num_lines (
```

```
struct gpiod_chip_info * info )
```

Get the number of lines exposed by the chip.

Parameters

<i>info</i>	GPIO chip info object.
-------------	------------------------

Returns

Number of GPIO lines.

4.3 Line definitions

Enumerations

- enum `gpiod_line_value` { `GPIOD_LINE_VALUE_ERROR` = -1 , `GPIOD_LINE_VALUE_INACTIVE` = 0 , `GPIOD_LINE_VALUE_ACTIVE` = 1 }
Logical line state.
- enum `gpiod_line_direction` { `GPIOD_LINE_DIRECTION_AS_IS` = 1 , `GPIOD_LINE_DIRECTION_INPUT` , `GPIOD_LINE_DIRECTION_OUTPUT` }
Direction settings.
- enum `gpiod_line_edge` { `GPIOD_LINE_EDGE_NONE` = 1 , `GPIOD_LINE_EDGE_RISING` , `GPIOD_LINE_EDGE_FALLING` , `GPIOD_LINE_EDGE_BOTH` }
Edge detection settings.
- enum `gpiod_line_bias` { `GPIOD_LINE_BIAS_AS_IS` = 1 , `GPIOD_LINE_BIAS_UNKNOWN` , `GPIOD_LINE_BIAS_DISABLED` , `GPIOD_LINE_BIAS_PULL_UP` , `GPIOD_LINE_BIAS_PULL_DOWN` }
Internal bias settings.
- enum `gpiod_line_drive` { `GPIOD_LINE_DRIVE_PUSH_PULL` = 1 , `GPIOD_LINE_DRIVE_OPEN_DRAIN` , `GPIOD_LINE_DRIVE_OPEN_SOURCE` }
Drive settings.
- enum `gpiod_line_clock` { `GPIOD_LINE_CLOCK_MONOTONIC` = 1 , `GPIOD_LINE_CLOCK_REALTIME` , `GPIOD_LINE_CLOCK_HTE` }
Clock settings.

4.3.1 Detailed Description

These defines are used across the API.

4.3.2 Enumeration Type Documentation

4.3.2.1 `gpiod_line_bias`

```
enum gpiod_line_bias
```

Internal bias settings.

Enumerator

GPIO_LINE_BIAS_AS_IS	Don't change the bias setting when applying line config.
GPIO_LINE_BIAS_UNKNOWN	The internal bias state is unknown.
GPIO_LINE_BIAS_DISABLED	The internal bias is disabled.
GPIO_LINE_BIAS_PULL_UP	The internal pull-up bias is enabled.
GPIO_LINE_BIAS_PULL_DOWN	The internal pull-down bias is enabled.

4.3.2.2 gpiod_line_clock

```
enum gpiod_line_clock
```

Clock settings.

Enumerator

GPIO_LINE_CLOCK_MONOTONIC	Line uses the monotonic clock for edge event timestamps.
GPIO_LINE_CLOCK_REALTIME	Line uses the realtime clock for edge event timestamps.
GPIO_LINE_CLOCK_HTE	Line uses the hardware timestamp engine for event timestamps.

4.3.2.3 gpiod_line_direction

```
enum gpiod_line_direction
```

Direction settings.

Enumerator

GPIO_LINE_DIRECTION_AS_IS	Request the line(s), but don't change direction.
GPIO_LINE_DIRECTION_INPUT	Direction is input - for reading the value of an externally driven GPIO line.
GPIO_LINE_DIRECTION_OUTPUT	Direction is output - for driving the GPIO line.

4.3.2.4 gpiod_line_drive

```
enum gpiod_line_drive
```

Drive settings.

Enumerator

GPIO_LINE_DRIVE_PUSH_PULL	Drive setting is push-pull.
GPIO_LINE_DRIVE_OPEN_DRAIN	Line output is open-drain.
GPIO_LINE_DRIVE_OPEN_SOURCE	Line output is open-source.

4.3.2.5 `gpiod_line_edge`

enum `gpiod_line_edge`

Edge detection settings.

Enumerator

<code>GPIOD_LINE_EDGE_NONE</code>	Line edge detection is disabled.
<code>GPIOD_LINE_EDGE_RISING</code>	Line detects rising edge events.
<code>GPIOD_LINE_EDGE_FALLING</code>	Line detects falling edge events.
<code>GPIOD_LINE_EDGE_BOTH</code>	Line detects both rising and falling edge events.

4.3.2.6 `gpiod_line_value`

enum `gpiod_line_value`

Logical line state.

Enumerator

<code>GPIOD_LINE_VALUE_ERROR</code>	Returned to indicate an error when reading the value.
<code>GPIOD_LINE_VALUE_INACTIVE</code>	Line is logically inactive.
<code>GPIOD_LINE_VALUE_ACTIVE</code>	Line is logically active.

4.4 Line info

Functions

- void `gpiod_line_info_free` (struct `gpiod_line_info` *info)
Free a line info object and release all associated resources.
- struct `gpiod_line_info` * `gpiod_line_info_copy` (struct `gpiod_line_info` *info)
Copy a line info object.
- unsigned int `gpiod_line_info_get_offset` (struct `gpiod_line_info` *info)
Get the offset of the line.
- const char * `gpiod_line_info_get_name` (struct `gpiod_line_info` *info)
Get the name of the line.
- bool `gpiod_line_info_is_used` (struct `gpiod_line_info` *info)
Check if the line is in use.
- const char * `gpiod_line_info_get_consumer` (struct `gpiod_line_info` *info)
Get the name of the consumer of the line.
- enum `gpiod_line_direction` `gpiod_line_info_get_direction` (struct `gpiod_line_info` *info)
Get the direction setting of the line.
- enum `gpiod_line_edge` `gpiod_line_info_get_edge_detection` (struct `gpiod_line_info` *info)
Get the edge detection setting of the line.
- enum `gpiod_line_bias` `gpiod_line_info_get_bias` (struct `gpiod_line_info` *info)

Get the bias setting of the line.

- enum `gpiod_line_drive` `gpiod_line_info_get_drive` (struct `gpiod_line_info` *info)

Get the drive setting of the line.

- bool `gpiod_line_info_is_active_low` (struct `gpiod_line_info` *info)

Check if the logical value of the line is inverted compared to the physical.

- bool `gpiod_line_info_is_debounced` (struct `gpiod_line_info` *info)

Check if the line is debounced (either by hardware or by the kernel software debouncer).

- unsigned long `gpiod_line_info_get_debounce_period_us` (struct `gpiod_line_info` *info)

Get the debounce period of the line, in microseconds.

- enum `gpiod_line_clock` `gpiod_line_info_get_event_clock` (struct `gpiod_line_info` *info)

Get the event clock setting used for edge event timestamps for the line.

4.4.1 Detailed Description

Functions for retrieving kernel information about both requested and free lines.

Line info object contains an immutable snapshot of a line's status.

The line info contains all the publicly available information about a line, which does not include the line value. The line must be requested to access the line value.

Some accessor methods return pointers. Those pointers refer to internal fields. The lifetimes of those fields are tied to the lifetime of the containing line info object. Such pointers remain valid until `gpiod_line_info_free` is called on the containing line info object. They must not be freed by the caller.

4.4.2 Function Documentation

4.4.2.1 `gpiod_line_info_copy()`

```
struct gpiod_line_info * gpiod_line_info_copy (
    struct gpiod_line_info * info )
```

Copy a line info object.

Parameters

<code>info</code>	Line info to copy.
-------------------	--------------------

Returns

Copy of the line info or NULL on error. The returned object must be freed by the caller using `:gpiod_line_info_free`.

4.4.2.2 `gpiod_line_info_free()`

```
void gpiod_line_info_free (
    struct gpiod_line_info * info )
```

Free a line info object and release all associated resources.

Parameters

<i>info</i>	GPIO line info object to free.
-------------	--------------------------------

4.4.2.3 `gpiod_line_info_get_bias()`

```
enum gpiod\_line\_bias gpiod_line_info_get_bias (
    struct gpiod\_line\_info * info )
```

Get the bias setting of the line.

Parameters

<i>info</i>	GPIO line object.
-------------	-------------------

Returns

Returns [GPIOD_LINE_BIAS_PULL_UP](#), [GPIOD_LINE_BIAS_PULL_DOWN](#), [GPIOD_LINE_BIAS_DISABLED](#) or [GPIOD_LINE_BIAS_UNKNOWN](#).

4.4.2.4 `gpiod_line_info_get_consumer()`

```
const char * gpiod_line_info_get_consumer (
    struct gpiod\_line\_info * info )
```

Get the name of the consumer of the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Name of the GPIO consumer as it is represented in the kernel. This function returns a valid pointer to a null-terminated string or NULL if the consumer name is not set. The string lifetime is tied to the line info object so the pointer must not be freed.

4.4.2.5 `gpiod_line_info_get_debounce_period_us()`

```
unsigned long gpiod_line_info_get_debounce_period_us (
    struct gpiod\_line\_info * info )
```

Get the debounce period of the line, in microseconds.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Debounce period in microseconds. 0 if the line is not debounced.

4.4.2.6 `gpiod_line_info_get_direction()`

```
enum gpiod_line_direction gpiod_line_info_get_direction (
    struct gpiod_line_info * info )
```

Get the direction setting of the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Returns `GPIO_LINE_DIRECTION_INPUT` or `GPIO_LINE_DIRECTION_OUTPUT`.

4.4.2.7 `gpiod_line_info_get_drive()`

```
enum gpiod_line_drive gpiod_line_info_get_drive (
    struct gpiod_line_info * info )
```

Get the drive setting of the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Returns `GPIO_LINE_DRIVE_PUSH_PULL`, `GPIO_LINE_DRIVE_OPEN_DRAIN` or `GPIO_LINE_DRIVE_OPEN_SOURCE`.

4.4.2.8 `gpiod_line_info_get_edge_detection()`

```
enum gpiod_line_edge gpiod_line_info_get_edge_detection (
    struct gpiod_line_info * info )
```

Get the edge detection setting of the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Returns [GPIOD_LINE_EDGE_NONE](#), [GPIOD_LINE_EDGE_RISING](#), [GPIOD_LINE_EDGE_FALLING](#) or [GPIOD_LINE_EDGE_BOTH](#).

4.4.2.9 `gpiod_line_info_get_event_clock()`

```
enum gpiod\_line\_clock gpiod_line_info_get_event_clock (
    struct gpiod\_line\_info * info )
```

Get the event clock setting used for edge event timestamps for the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Returns [GPIOD_LINE_CLOCK_MONOTONIC](#), [GPIOD_LINE_CLOCK_HTE](#) or [GPIOD_LINE_CLOCK_REALTIME](#).

4.4.2.10 `gpiod_line_info_get_name()`

```
const char * gpiod_line_info_get_name (
    struct gpiod\_line\_info * info )
```

Get the name of the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Name of the GPIO line as it is represented in the kernel. This function returns a valid pointer to a null-terminated string or NULL if the line is unnamed. The string lifetime is tied to the line info object so the pointer must not be freed.

4.4.2.11 `gpiod_line_info_get_offset()`

```
unsigned int gpiod_line_info_get_offset (
    struct gpiod\_line\_info * info )
```

Get the offset of the line.

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

Offset of the line within the parent chip.

The offset uniquely identifies the line on the chip. The combination of the chip and offset uniquely identifies the line within the system.

4.4.2.12 `gpio_line_info_is_active_low()`

```
bool gpio_line_info_is_active_low (
    struct gpio_line_info * info )
```

Check if the logical value of the line is inverted compared to the physical.

Parameters

<i>info</i>	GPIO line object.
-------------	-------------------

Returns

True if the line is "active-low", false otherwise.

4.4.2.13 `gpio_line_info_is_debounced()`

```
bool gpio_line_info_is_debounced (
    struct gpio_line_info * info )
```

Check if the line is debounced (either by hardware or by the kernel software debouncer).

Parameters

<i>info</i>	GPIO line info object.
-------------	------------------------

Returns

True if the line is debounced, false otherwise.

4.4.2.14 `gpio_line_info_is_used()`

```
bool gpio_line_info_is_used (
    struct gpio_line_info * info )
```

Check if the line is in use.

Parameters

<i>info</i>	GPIO line object.
-------------	-------------------

Returns

True if the line is in use, false otherwise.

The exact reason a line is busy cannot be determined from user space. It may have been requested by another process or hogged by the kernel. It only matters that the line is used and can't be requested until released by the existing consumer.

4.5 Line status watch events

Enumerations

- enum `gpiod_info_event_type` { `GPIOD_INFO_EVENT_LINE_REQUESTED` = 1 , `GPIOD_INFO_EVENT_LINE_RELEASED` , `GPIOD_INFO_EVENT_LINE_CONFIG_CHANGED` }

Line status change event types.

Functions

- void `gpiod_info_event_free` (struct `gpiod_info_event` *event)
Free the info event object and release all associated resources.
- enum `gpiod_info_event_type` `gpiod_info_event_get_event_type` (struct `gpiod_info_event` *event)
Get the event type of the status change event.
- uint64_t `gpiod_info_event_get_timestamp_ns` (struct `gpiod_info_event` *event)
Get the timestamp of the event.
- struct `gpiod_line_info` * `gpiod_info_event_get_line_info` (struct `gpiod_info_event` *event)
Get the snapshot of line-info associated with the event.

4.5.1 Detailed Description

Accessors for the info event objects allowing to monitor changes in GPIO line status.

Callers are notified about changes in a line's status due to GPIO uAPI calls. Each info event contains information about the event itself (timestamp, type) as well as a snapshot of line's status in the form of a line-info object.

4.5.2 Enumeration Type Documentation

4.5.2.1 `gpiod_info_event_type`

enum `gpiod_info_event_type`

Line status change event types.

Enumerator

<code>GPIOD_INFO_EVENT_LINE_REQUESTED</code>	Line has been requested.
<code>GPIOD_INFO_EVENT_LINE_RELEASED</code>	Previously requested line has been released.
<code>GPIOD_INFO_EVENT_LINE_CONFIG_CHANGED</code>	Line configuration has changed.

4.5.3 Function Documentation

4.5.3.1 `gpiod_info_event_free()`

```
void gpiod_info_event_free (
    struct gpiod\_info\_event * event )
```

Free the info event object and release all associated resources.

Parameters

<i>event</i>	Info event to free.
--------------	---------------------

4.5.3.2 `gpiod_info_event_get_event_type()`

```
enum gpiod\_info\_event\_type gpiod_info_event_get_event_type (
    struct gpiod\_info\_event * event )
```

Get the event type of the status change event.

Parameters

<i>event</i>	Line status watch event.
--------------	--------------------------

Returns

One of [GPIOD_INFO_EVENT_LINE_REQUESTED](#), [GPIOD_INFO_EVENT_LINE_RELEASED](#) or [GPIOD_INFO_EVENT_LINE](#)

4.5.3.3 `gpiod_info_event_get_line_info()`

```
struct gpiod\_line\_info * gpiod_info_event_get_line_info (
    struct gpiod\_info\_event * event )
```

Get the snapshot of line-info associated with the event.

Parameters

<i>event</i>	Line info event object.
--------------	-------------------------

Returns

Returns a pointer to the line-info object associated with the event. The object lifetime is tied to the event object, so the pointer must not be freed by the caller.

Warning

Thread-safety: Since the line-info object is tied to the event, different threads may not operate on the event and line-info at the same time. The line-info can be copied using [gpiod_line_info_copy](#) in order to create a standalone object - which then may safely be used from a different thread concurrently.

4.5.3.4 `gpiod_info_event_get_timestamp_ns()`

```
uint64_t gpiod_info_event_get_timestamp_ns (
    struct gpiod_info_event * event )
```

Get the timestamp of the event.

Parameters

<code>event</code>	Line status watch event.
--------------------	--------------------------

Returns

Timestamp in nanoseconds, read from the monotonic clock.

4.6 Line settings objects

Functions

- struct `gpiod_line_settings` * `gpiod_line_settings_new` (void)
Create a new line settings object.
- void `gpiod_line_settings_free` (struct `gpiod_line_settings` *settings)
Free the line settings object and release all associated resources.
- void `gpiod_line_settings_reset` (struct `gpiod_line_settings` *settings)
Reset the line settings object to its default values.
- struct `gpiod_line_settings` * `gpiod_line_settings_copy` (struct `gpiod_line_settings` *settings)
Copy the line settings object.
- int `gpiod_line_settings_set_direction` (struct `gpiod_line_settings` *settings, enum `gpiod_line_direction` direction)
Set direction.
- enum `gpiod_line_direction` `gpiod_line_settings_get_direction` (struct `gpiod_line_settings` *settings)
Get direction.
- int `gpiod_line_settings_set_edge_detection` (struct `gpiod_line_settings` *settings, enum `gpiod_line_edge` edge)
Set edge detection.
- enum `gpiod_line_edge` `gpiod_line_settings_get_edge_detection` (struct `gpiod_line_settings` *settings)
Get edge detection.
- int `gpiod_line_settings_set_bias` (struct `gpiod_line_settings` *settings, enum `gpiod_line_bias` bias)
Set bias.
- enum `gpiod_line_bias` `gpiod_line_settings_get_bias` (struct `gpiod_line_settings` *settings)
Get bias.
- int `gpiod_line_settings_set_drive` (struct `gpiod_line_settings` *settings, enum `gpiod_line_drive` drive)
Set drive.
- enum `gpiod_line_drive` `gpiod_line_settings_get_drive` (struct `gpiod_line_settings` *settings)
Get drive.
- void `gpiod_line_settings_set_active_low` (struct `gpiod_line_settings` *settings, bool active_low)
Set active-low setting.
- bool `gpiod_line_settings_get_active_low` (struct `gpiod_line_settings` *settings)
Get active-low setting.

- void `gpiod_line_settings_set_debounce_period_us` (struct `gpiod_line_settings` *settings, unsigned long period)
Set debounce period.
- unsigned long `gpiod_line_settings_get_debounce_period_us` (struct `gpiod_line_settings` *settings)
Get debounce period.
- int `gpiod_line_settings_set_event_clock` (struct `gpiod_line_settings` *settings, enum `gpiod_line_clock` event↔_clock)
Set event clock.
- enum `gpiod_line_clock` `gpiod_line_settings_get_event_clock` (struct `gpiod_line_settings` *settings)
Get event clock setting.
- int `gpiod_line_settings_set_output_value` (struct `gpiod_line_settings` *settings, enum `gpiod_line_value` value)
Set the output value.
- enum `gpiod_line_value` `gpiod_line_settings_get_output_value` (struct `gpiod_line_settings` *settings)
Get the output value.

4.6.1 Detailed Description

Functions for manipulating line settings objects.

Line settings object contains a set of line properties that can be used when requesting lines or reconfiguring an existing request.

Mutators in general can only fail if the new property value is invalid. The return values can be safely ignored - the object remains valid even after a mutator fails and simply uses the sane default appropriate for given property.

4.6.2 Function Documentation

4.6.2.1 `gpiod_line_settings_copy()`

```
struct gpiod_line_settings * gpiod_line_settings_copy (
    struct gpiod_line_settings * settings )
```

Copy the line settings object.

Parameters

<code>settings</code>	Line settings object to copy.
-----------------------	-------------------------------

Returns

New line settings object that must be freed using `gpiod_line_settings_free` or NULL on failure.

4.6.2.2 `gpiod_line_settings_free()`

```
void gpiod_line_settings_free (
    struct gpiod_line_settings * settings )
```

Free the line settings object and release all associated resources.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

4.6.2.3 `gpiod_line_settings_get_active_low()`

```
bool gpiod_line_settings_get_active_low (
    struct gpiod_line_settings * settings )
```

Get active-low setting.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

True if active-low is enabled, false otherwise.

4.6.2.4 `gpiod_line_settings_get_bias()`

```
enum gpiod_line_bias gpiod_line_settings_get_bias (
    struct gpiod_line_settings * settings )
```

Get bias.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current bias setting.

4.6.2.5 `gpiod_line_settings_get_debounce_period_us()`

```
unsigned long gpiod_line_settings_get_debounce_period_us (
    struct gpiod_line_settings * settings )
```

Get debounce period.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current debounce period in microseconds.

4.6.2.6 `gpiod_line_settings_get_direction()`

```
enum gpiod_line_direction gpiod_line_settings_get_direction (
    struct gpiod_line_settings * settings )
```

Get direction.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current direction.

4.6.2.7 `gpiod_line_settings_get_drive()`

```
enum gpiod_line_drive gpiod_line_settings_get_drive (
    struct gpiod_line_settings * settings )
```

Get drive.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current drive setting.

4.6.2.8 `gpiod_line_settings_get_edge_detection()`

```
enum gpiod_line_edge gpiod_line_settings_get_edge_detection (
    struct gpiod_line_settings * settings )
```

Get edge detection.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current edge detection setting.

4.6.2.9 `gpiod_line_settings_get_event_clock()`

```
enum gpiod\_line\_clock gpiod_line_settings_get_event_clock (
    struct gpiod\_line\_settings * settings )
```

Get event clock setting.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current event clock setting.

4.6.2.10 `gpiod_line_settings_get_output_value()`

```
enum gpiod\_line\_value gpiod_line_settings_get_output_value (
    struct gpiod\_line\_settings * settings )
```

Get the output value.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

Returns

Current output value.

4.6.2.11 `gpiod_line_settings_new()`

```
struct gpiod\_line\_settings * gpiod_line_settings_new (
    void )
```

Create a new line settings object.

Returns

New line settings object or NULL on error. The returned object must be freed by the caller using [gpiod_line_settings_free](#).

4.6.2.12 `gpiod_line_settings_reset()`

```
void gpiod_line_settings_reset (
    struct gpiod_line_settings * settings )
```

Reset the line settings object to its default values.

Parameters

<i>settings</i>	Line settings object.
-----------------	-----------------------

4.6.2.13 `gpiod_line_settings_set_active_low()`

```
void gpiod_line_settings_set_active_low (
    struct gpiod_line_settings * settings,
    bool active_low )
```

Set active-low setting.

Parameters

<i>settings</i>	Line settings object.
<i>active_low</i>	New active-low setting.

4.6.2.14 `gpiod_line_settings_set_bias()`

```
int gpiod_line_settings_set_bias (
    struct gpiod_line_settings * settings,
    enum gpiod_line_bias bias )
```

Set bias.

Parameters

<i>settings</i>	Line settings object.
<i>bias</i>	New bias.

Returns

0 on success, -1 on failure.

4.6.2.15 `gpiod_line_settings_set_debounce_period_us()`

```
void gpiod_line_settings_set_debounce_period_us (
    struct gpiod_line_settings * settings,
    unsigned long period )
```

Set debounce period.

Parameters

<i>settings</i>	Line settings object.
<i>period</i>	New debounce period in microseconds.

4.6.2.16 `gpiod_line_settings_set_direction()`

```
int gpiod_line_settings_set_direction (
    struct gpiod_line_settings * settings,
    enum gpiod_line_direction direction )
```

Set direction.

Parameters

<i>settings</i>	Line settings object.
<i>direction</i>	New direction.

Returns

0 on success, -1 on error.

4.6.2.17 `gpiod_line_settings_set_drive()`

```
int gpiod_line_settings_set_drive (
    struct gpiod_line_settings * settings,
    enum gpiod_line_drive drive )
```

Set drive.

Parameters

<i>settings</i>	Line settings object.
<i>drive</i>	New drive setting.

Returns

0 on success, -1 on failure.

4.6.2.18 `gpiod_line_settings_set_edge_detection()`

```
int gpiod_line_settings_set_edge_detection (
    struct gpiod_line_settings * settings,
    enum gpiod_line_edge edge )
```

Set edge detection.

Parameters

<i>settings</i>	Line settings object.
<i>edge</i>	New edge detection setting.

Returns

0 on success, -1 on failure.

4.6.2.19 `gpiod_line_settings_set_event_clock()`

```
int gpiod_line_settings_set_event_clock (
    struct gpiod\_line\_settings * settings,
    enum gpiod\_line\_clock event_clock )
```

Set event clock.

Parameters

<i>settings</i>	Line settings object.
<i>event_clock</i>	New event clock.

Returns

0 on success, -1 on failure.

4.6.2.20 `gpiod_line_settings_set_output_value()`

```
int gpiod_line_settings_set_output_value (
    struct gpiod\_line\_settings * settings,
    enum gpiod\_line\_value value )
```

Set the output value.

Parameters

<i>settings</i>	Line settings object.
<i>value</i>	New output value.

Returns

0 on success, -1 on failure.

4.7 Line configuration objects

Functions

- struct [gpiod_line_config](#) * [gpiod_line_config_new](#) (void)
Create a new line config object.
- void [gpiod_line_config_free](#) (struct [gpiod_line_config](#) *config)
Free the line config object and release all associated resources.
- void [gpiod_line_config_reset](#) (struct [gpiod_line_config](#) *config)

Reset the line config object.

- int [gpiod_line_config_add_line_settings](#) (struct [gpiod_line_config](#) *config, const unsigned int *offsets, size_t num_offsets, struct [gpiod_line_settings](#) *settings)

Add line settings for a set of offsets.

- struct [gpiod_line_settings](#) * [gpiod_line_config_get_line_settings](#) (struct [gpiod_line_config](#) *config, unsigned int offset)

Get line settings for offset.

- int [gpiod_line_config_set_output_values](#) (struct [gpiod_line_config](#) *config, const enum [gpiod_line_value](#) *values, size_t num_values)

Set output values for a number of lines.

- size_t [gpiod_line_config_get_num_configured_offsets](#) (struct [gpiod_line_config](#) *config)

Get the number of configured line offsets.

- size_t [gpiod_line_config_get_configured_offsets](#) (struct [gpiod_line_config](#) *config, unsigned int *offsets, size_t max_offsets)

Get configured offsets.

4.7.1 Detailed Description

Functions for manipulating line configuration objects.

The line-config object contains the configuration for lines that can be used in two cases:

- when making a line request
- when reconfiguring a set of already requested lines.

A new line-config object is empty. Using it in a request will lead to an error. In order to a line-config to become useful, it needs to be assigned at least one offset-to-settings mapping by calling [gpiod_line_config_add_line_settings](#).

When calling [gpiod_chip_request_lines](#), the library will request all offsets that were assigned settings in the order that they were assigned. If any of the offsets was duplicated, the last one will take precedence.

4.7.2 Function Documentation

4.7.2.1 [gpiod_line_config_add_line_settings\(\)](#)

```
int gpiod_line_config_add_line_settings (
    struct gpiod\_line\_config * config,
    const unsigned int * offsets,
    size_t num_offsets,
    struct gpiod\_line\_settings * settings )
```

Add line settings for a set of offsets.

Parameters

<i>config</i>	Line config object.
<i>offsets</i>	Array of offsets for which to apply the settings.
<i>num_offsets</i>	Number of offsets stored in the offsets array.
<i>settings</i>	Line settings to apply.

Returns

0 on success, -1 on failure.

4.7.2.2 gpiod_line_config_free()

```
void gpiod_line_config_free (
    struct gpiod_line_config * config )
```

Free the line config object and release all associated resources.

Parameters

<i>config</i>	Line config object to free.
---------------	-----------------------------

4.7.2.3 gpiod_line_config_get_configured_offsets()

```
size_t gpiod_line_config_get_configured_offsets (
    struct gpiod_line_config * config,
    unsigned int * offsets,
    size_t max_offsets )
```

Get configured offsets.

Parameters

<i>config</i>	Line config object.
<i>offsets</i>	Array to store offsets.
<i>max_offsets</i>	Number of offsets that can be stored in the offsets array.

Returns

Number of offsets stored in the offsets array.

If *max_offsets* is lower than the number of lines actually requested (this value can be retrieved using [gpiod_line_config_get_num_configured_offsets](#)), then only up to *max_lines* offsets will be stored in offsets.

4.7.2.4 gpiod_line_config_get_line_settings()

```
struct gpiod_line_settings * gpiod_line_config_get_line_settings (
    struct gpiod_line_config * config,
    unsigned int offset )
```

Get line settings for offset.

Parameters

<i>config</i>	Line config object.
<i>offset</i>	Offset for which to get line settings.

Returns

New line settings object (must be freed by the caller) or NULL on error.

4.7.2.5 `gpiod_line_config_get_num_configured_offsets()`

```
size_t gpiod_line_config_get_num_configured_offsets (
    struct gpiod\_line\_config * config )
```

Get the number of configured line offsets.

Parameters

<i>config</i>	Line config object.
---------------	---------------------

Returns

Number of offsets for which line settings have been added.

4.7.2.6 `gpiod_line_config_new()`

```
struct gpiod\_line\_config * gpiod_line_config_new (
    void )
```

Create a new line config object.

Returns

New line config object or NULL on error. The returned object must be freed by the caller using [gpiod_line_config_free](#).

4.7.2.7 `gpiod_line_config_reset()`

```
void gpiod_line_config_reset (
    struct gpiod\_line\_config * config )
```

Reset the line config object.

Parameters

<i>config</i>	Line config object to free.
---------------	-----------------------------

Resets the entire configuration stored in the object. This is useful if the user wants to reuse the object without reallocating it.

4.7.2.8 `gpiod_line_config_set_output_values()`

```
int gpiod_line_config_set_output_values (
```

```
struct gpiod_line_config * config,
const enum gpiod_line_value * values,
size_t num_values )
```

Set output values for a number of lines.

Parameters

<i>config</i>	Line config object.
<i>values</i>	Buffer containing the output values.
<i>num_values</i>	Number of values in the buffer.

Returns

0 on success, -1 on error.

This is a helper that allows users to set multiple (potentially different) output values at once while using the same line settings object. Instead of modifying the output value in the settings object and calling [gpiod_line_config_add_line_settings](#) multiple times, we can specify the settings, add them for a set of offsets and then call this function to set the output values.

Values set by this function override whatever values were specified in the regular line settings.

Each value must be associated with the line identified by the corresponding entry in the offset array filled by [gpiod_line_request_get_requested_offsets](#).

4.8 Request configuration objects

Functions

- struct [gpiod_request_config](#) * [gpiod_request_config_new](#) (void)
Create a new request config object.
- void [gpiod_request_config_free](#) (struct [gpiod_request_config](#) *config)
Free the request config object and release all associated resources.
- void [gpiod_request_config_set_consumer](#) (struct [gpiod_request_config](#) *config, const char *consumer)
Set the consumer name for the request.
- const char * [gpiod_request_config_get_consumer](#) (struct [gpiod_request_config](#) *config)
Get the consumer name configured in the request config.
- void [gpiod_request_config_set_event_buffer_size](#) (struct [gpiod_request_config](#) *config, size_t event_buffer↵_size)
Set the size of the kernel event buffer for the request.
- size_t [gpiod_request_config_get_event_buffer_size](#) (struct [gpiod_request_config](#) *config)
Get the edge event buffer size for the request config.

4.8.1 Detailed Description

Functions for manipulating request configuration objects.

Request config objects are used to pass a set of options to the kernel at the time of the line request. The mutators don't return error values. If the values are invalid, in general they are silently adjusted to acceptable ranges.

4.8.2 Function Documentation

4.8.2.1 `gpiod_request_config_free()`

```
void gpiod_request_config_free (
    struct gpiod_request_config * config )
```

Free the request config object and release all associated resources.

Parameters

<i>config</i>	Line config object.
---------------	---------------------

4.8.2.2 `gpiod_request_config_get_consumer()`

```
const char * gpiod_request_config_get_consumer (
    struct gpiod_request_config * config )
```

Get the consumer name configured in the request config.

Parameters

<i>config</i>	Request config object.
---------------	------------------------

Returns

Consumer name stored in the request config.

4.8.2.3 `gpiod_request_config_get_event_buffer_size()`

```
size_t gpiod_request_config_get_event_buffer_size (
    struct gpiod_request_config * config )
```

Get the edge event buffer size for the request config.

Parameters

<i>config</i>	Request config object.
---------------	------------------------

Returns

Edge event buffer size setting from the request config.

4.8.2.4 `gpiod_request_config_new()`

```
struct gpiod_request_config * gpiod_request_config_new (
    void )
```

Create a new request config object.

Returns

New request config object or NULL on error. The returned object must be freed by the caller using [gpiod_request_config_free](#).

4.8.2.5 gpiod_request_config_set_consumer()

```
void gpiod_request_config_set_consumer (
    struct gpiod\_request\_config * config,
    const char * consumer )
```

Set the consumer name for the request.

Parameters

<i>config</i>	Request config object.
<i>consumer</i>	Consumer name.

Note

If the consumer string is too long, it will be truncated to the max accepted length.

4.8.2.6 gpiod_request_config_set_event_buffer_size()

```
void gpiod_request_config_set_event_buffer_size (
    struct gpiod\_request\_config * config,
    size_t event_buffer_size )
```

Set the size of the kernel event buffer for the request.

Parameters

<i>config</i>	Request config object.
<i>event_buffer_size</i>	New event buffer size.

Note

The kernel may adjust the value if it's too high. If set to 0, the default value will be used.

The kernel buffer is distinct from and independent of the user space buffer ([gpiod_edge_event_buffer_new](#)).

4.9 Line request operations**Functions**

- void [gpiod_line_request_release](#) (struct [gpiod_line_request](#) *request)
Release the requested lines and free all associated resources.
- const char * [gpiod_line_request_get_chip_name](#) (struct [gpiod_line_request](#) *request)

- Get the name of the chip this request was made on.*

 - `size_t gpiod_line_request_get_num_requested_lines` (struct `gpiod_line_request` *request)

Get the number of lines in the request.
- `size_t gpiod_line_request_get_requested_offsets` (struct `gpiod_line_request` *request, unsigned int *offsets, size_t max_offsets)

Get the offsets of the lines in the request.
- `enum gpiod_line_value gpiod_line_request_get_value` (struct `gpiod_line_request` *request, unsigned int offset)

Get the value of a single requested line.
- `int gpiod_line_request_get_values_subset` (struct `gpiod_line_request` *request, size_t num_values, const unsigned int *offsets, enum `gpiod_line_value` *values)

Get the values of a subset of requested lines.
- `int gpiod_line_request_get_values` (struct `gpiod_line_request` *request, enum `gpiod_line_value` *values)

Get the values of all requested lines.
- `int gpiod_line_request_set_value` (struct `gpiod_line_request` *request, unsigned int offset, enum `gpiod_line_value` value)

Set the value of a single requested line.
- `int gpiod_line_request_set_values_subset` (struct `gpiod_line_request` *request, size_t num_values, const unsigned int *offsets, const enum `gpiod_line_value` *values)

Set the values of a subset of requested lines.
- `int gpiod_line_request_set_values` (struct `gpiod_line_request` *request, const enum `gpiod_line_value` *values)

Set the values of all lines associated with a request.
- `int gpiod_line_request_reconfigure_lines` (struct `gpiod_line_request` *request, struct `gpiod_line_config` *config)

Update the configuration of lines associated with a line request.
- `int gpiod_line_request_get_fd` (struct `gpiod_line_request` *request)

Get the file descriptor associated with a line request.
- `int gpiod_line_request_wait_edge_events` (struct `gpiod_line_request` *request, int64_t timeout_ns)

Wait for edge events on any of the requested lines.
- `int gpiod_line_request_read_edge_events` (struct `gpiod_line_request` *request, struct `gpiod_edge_event_buffer` *buffer, size_t max_events)

Read a number of edge events from a line request.

4.9.1 Detailed Description

Functions allowing interactions with requested lines.

4.9.2 Function Documentation

4.9.2.1 gpiod_line_request_get_chip_name()

```
const char * gpiod_line_request_get_chip_name (
    struct gpiod_line_request * request )
```

Get the name of the chip this request was made on.

Parameters

<i>request</i>	Line request object.
----------------	----------------------

Returns

Name the GPIO chip device. The returned pointer is valid for the lifetime of the request object and must not be freed by the caller.

4.9.2.2 gpiod_line_request_get_fd()

```
int gpiod_line_request_get_fd (
    struct gpiod_line_request * request )
```

Get the file descriptor associated with a line request.

Parameters

<i>request</i>	GPIO line request.
----------------	--------------------

Returns

The file descriptor associated with the request. This function never fails. The returned file descriptor must not be closed by the caller. Call [gpiod_line_request_release](#) to close the file.

4.9.2.3 gpiod_line_request_get_num_requested_lines()

```
size_t gpiod_line_request_get_num_requested_lines (
    struct gpiod_line_request * request )
```

Get the number of lines in the request.

Parameters

<i>request</i>	Line request object.
----------------	----------------------

Returns

Number of requested lines.

4.9.2.4 gpiod_line_request_get_requested_offsets()

```
size_t gpiod_line_request_get_requested_offsets (
    struct gpiod_line_request * request,
    unsigned int * offsets,
    size_t max_offsets )
```

Get the offsets of the lines in the request.

Parameters

<i>request</i>	Line request object.
<i>offsets</i>	Array to store offsets.
<i>max_offsets</i>	Number of offsets that can be stored in the offsets array.

Returns

Number of offsets stored in the offsets array.

If `max_offsets` is lower than the number of lines actually requested (this value can be retrieved using [gpiod_line_request_get_num_requested_lines](#)), then only up to `max_lines` offsets will be stored in offsets.

4.9.2.5 gpiod_line_request_get_value()

```
enum gpiod_line_value gpiod_line_request_get_value (
    struct gpiod_line_request * request,
    unsigned int offset )
```

Get the value of a single requested line.

Parameters

<i>request</i>	Line request object.
<i>offset</i>	The offset of the line of which the value should be read.

Returns

Returns 1 or 0 on success and -1 on error.

4.9.2.6 gpiod_line_request_get_values()

```
int gpiod_line_request_get_values (
    struct gpiod_line_request * request,
    enum gpiod_line_value * values )
```

Get the values of all requested lines.

Parameters

<i>request</i>	GPIO line request.
<i>values</i>	Array in which the values will be stored. Must be sized to hold the number of lines filled by gpiod_line_request_get_num_requested_lines . Each value is associated with the line identified by the corresponding entry in the offset array filled by gpiod_line_request_get_requested_offsets .

Returns

0 on success, -1 on failure.

4.9.2.7 gpiod_line_request_get_values_subset()

```
int gpiod_line_request_get_values_subset (
    struct gpiod_line_request * request,
```

```
size_t num_values,
const unsigned int * offsets,
enum gpiod_line_value * values )
```

Get the values of a subset of requested lines.

Parameters

<i>request</i>	GPIO line request.
<i>num_values</i>	Number of lines for which to read values.
<i>offsets</i>	Array of offsets identifying the subset of requested lines from which to read values.
<i>values</i>	Array in which the values will be stored. Must be sized to hold <code>num_values</code> entries. Each value is associated with the line identified by the corresponding entry in <code>offsets</code> .

Returns

0 on success, -1 on failure.

4.9.2.8 gpiod_line_request_read_edge_events()

```
int gpiod_line_request_read_edge_events (
    struct gpiod_line_request * request,
    struct gpiod_edge_event_buffer * buffer,
    size_t max_events )
```

Read a number of edge events from a line request.

Parameters

<i>request</i>	GPIO line request.
<i>buffer</i>	Edge event buffer, sized to hold at least <code>max_events</code> .
<i>max_events</i>	Maximum number of events to read.

Returns

On success returns the number of events read from the file descriptor, on failure return -1.

Note

This function will block if no event was queued for the line request.
Any existing events in the buffer are overwritten. This is not an append operation.

4.9.2.9 gpiod_line_request_reconfigure_lines()

```
int gpiod_line_request_reconfigure_lines (
    struct gpiod_line_request * request,
    struct gpiod_line_config * config )
```

Update the configuration of lines associated with a line request.

Parameters

<i>request</i>	GPIO line request.
<i>config</i>	New line config to apply.

Returns

0 on success, -1 on failure.

Note

The new line configuration completely replaces the old.

Any requested lines without overrides are configured to the requested defaults.

Any configured overrides for lines that have not been requested are silently ignored.

4.9.2.10 gpiod_line_request_release()

```
void gpiod_line_request_release (
    struct gpiod_line_request * request )
```

Release the requested lines and free all associated resources.

Parameters

<i>request</i>	Line request object to release.
----------------	---------------------------------

4.9.2.11 gpiod_line_request_set_value()

```
int gpiod_line_request_set_value (
    struct gpiod_line_request * request,
    unsigned int offset,
    enum gpiod_line_value value )
```

Set the value of a single requested line.

Parameters

<i>request</i>	Line request object.
<i>offset</i>	The offset of the line for which the value should be set.
<i>value</i>	Value to set.

Returns

0 on success, -1 on failure.

4.9.2.12 `gpiod_line_request_set_values()`

```
int gpiod_line_request_set_values (
    struct gpiod\_line\_request * request,
    const enum gpiod\_line\_value * values )
```

Set the values of all lines associated with a request.

Parameters

<i>request</i>	GPIO line request.
<i>values</i>	Array containing the values to set. Must be sized to contain the number of lines filled by gpiod_line_request_get_num_requested_lines . Each value is associated with the line identified by the corresponding entry in the offset array filled by gpiod_line_request_get_requested_offsets .

Returns

0 on success, -1 on failure.

4.9.2.13 `gpiod_line_request_set_values_subset()`

```
int gpiod_line_request_set_values_subset (
    struct gpiod\_line\_request * request,
    size_t num_values,
    const unsigned int * offsets,
    const enum gpiod\_line\_value * values )
```

Set the values of a subset of requested lines.

Parameters

<i>request</i>	GPIO line request.
<i>num_values</i>	Number of lines for which to set values.
<i>offsets</i>	Array of offsets, containing the number of entries specified by <i>num_values</i> , identifying the requested lines for which to set values.
<i>values</i>	Array of values to set, containing the number of entries specified by <i>num_values</i> . Each value is associated with the line identified by the corresponding entry in <i>offsets</i> .

Returns

0 on success, -1 on failure.

4.9.2.14 `gpiod_line_request_wait_edge_events()`

```
int gpiod_line_request_wait_edge_events (
    struct gpiod\_line\_request * request,
    int64_t timeout_ns )
```

Wait for edge events on any of the requested lines.

Parameters

<i>request</i>	GPIO line request.
<i>timeout_ns</i>	Wait time limit in nanoseconds. If set to 0, the function returns immediately. If set to a negative number, the function blocks indefinitely until an event becomes available.

Returns

0 if wait timed out, -1 if an error occurred, 1 if an event is pending.

Lines must have edge detection set for edge events to be emitted. By default edge detection is disabled.

4.10 Line edge events handling

Enumerations

- enum `gpiod_edge_event_type` { `GPIOD_EDGE_EVENT_RISING_EDGE` = 1, `GPIOD_EDGE_EVENT_FALLING_EDGE` }

Event types.

Functions

- void `gpiod_edge_event_free` (struct `gpiod_edge_event` *event)
Free the edge event object.
- struct `gpiod_edge_event` * `gpiod_edge_event_copy` (struct `gpiod_edge_event` *event)
Copy the edge event object.
- enum `gpiod_edge_event_type` `gpiod_edge_event_get_event_type` (struct `gpiod_edge_event` *event)
Get the event type.
- uint64_t `gpiod_edge_event_get_timestamp_ns` (struct `gpiod_edge_event` *event)
Get the timestamp of the event.
- unsigned int `gpiod_edge_event_get_line_offset` (struct `gpiod_edge_event` *event)
Get the offset of the line which triggered the event.
- unsigned long `gpiod_edge_event_get_global_seqno` (struct `gpiod_edge_event` *event)
Get the global sequence number of the event.
- unsigned long `gpiod_edge_event_get_line_seqno` (struct `gpiod_edge_event` *event)
Get the event sequence number specific to the line.
- struct `gpiod_edge_event_buffer` * `gpiod_edge_event_buffer_new` (size_t capacity)
Create a new edge event buffer.
- size_t `gpiod_edge_event_buffer_get_capacity` (struct `gpiod_edge_event_buffer` *buffer)
Get the capacity (the max number of events that can be stored) of the event buffer.
- void `gpiod_edge_event_buffer_free` (struct `gpiod_edge_event_buffer` *buffer)
Free the edge event buffer and release all associated resources.
- struct `gpiod_edge_event` * `gpiod_edge_event_buffer_get_event` (struct `gpiod_edge_event_buffer` *buffer, unsigned long index)
Get an event stored in the buffer.
- size_t `gpiod_edge_event_buffer_get_num_events` (struct `gpiod_edge_event_buffer` *buffer)
Get the number of events a buffer has stored.

4.10.1 Detailed Description

Functions and data types for handling edge events.

An edge event object contains information about a single line edge event. It contains the event type, timestamp and the offset of the line on which the event occurred as well as two sequence numbers (global for all lines in the associated request and local for this line only).

Edge events are stored into an edge-event buffer object to improve performance and to limit the number of memory allocations when a large number of events are being read.

4.10.2 Enumeration Type Documentation

4.10.2.1 `gpiod_edge_event_type`

```
enum gpiod_edge_event_type
```

Event types.

Enumerator

<code>GPIOD_EDGE_EVENT_RISING_EDGE</code>	Rising edge event.
<code>GPIOD_EDGE_EVENT_FALLING_EDGE</code>	Falling edge event.

4.10.3 Function Documentation

4.10.3.1 `gpiod_edge_event_buffer_free()`

```
void gpiod_edge_event_buffer_free (  
    struct gpiod_edge_event_buffer * buffer )
```

Free the edge event buffer and release all associated resources.

Parameters

<i>buffer</i>	Edge event buffer to free.
---------------	----------------------------

4.10.3.2 `gpiod_edge_event_buffer_get_capacity()`

```
size_t gpiod_edge_event_buffer_get_capacity (  
    struct gpiod_edge_event_buffer * buffer )
```

Get the capacity (the max number of events that can be stored) of the event buffer.

Parameters

<i>buffer</i>	Edge event buffer.
---------------	--------------------

Returns

The capacity of the buffer.

4.10.3.3 `gpiod_edge_event_buffer_get_event()`

```
struct gpiod\_edge\_event * gpiod_edge_event_buffer_get_event (
    struct gpiod\_edge\_event\_buffer * buffer,
    unsigned long index )
```

Get an event stored in the buffer.

Parameters

<i>buffer</i>	Edge event buffer.
<i>index</i>	Index of the event in the buffer.

Returns

Pointer to an event stored in the buffer. The lifetime of the event is tied to the buffer object. Users must not free the event returned by this function.

Warning

Thread-safety: Since events are tied to the buffer instance, different threads may not operate on the buffer and any associated events at the same time. Events can be copied using [gpiod_edge_event_copy](#) in order to create a standalone objects - which each may safely be used from a different thread concurrently.

4.10.3.4 `gpiod_edge_event_buffer_get_num_events()`

```
size_t gpiod_edge_event_buffer_get_num_events (
    struct gpiod\_edge\_event\_buffer * buffer )
```

Get the number of events a buffer has stored.

Parameters

<i>buffer</i>	Edge event buffer.
---------------	--------------------

Returns

Number of events stored in the buffer.

4.10.3.5 `gpiod_edge_event_buffer_new()`

```
struct gpiod\_edge\_event\_buffer * gpiod_edge_event_buffer_new (
    size_t capacity )
```

Create a new edge event buffer.

Parameters

<i>capacity</i>	Number of events the buffer can store (min = 1, max = 1024).
-----------------	--

Returns

New edge event buffer or NULL on error.

Note

If capacity equals 0, it will be set to a default value of 64. If capacity is larger than 1024, it will be limited to 1024.

The user space buffer is independent of the kernel buffer ([gpiod_request_config_set_event_buffer_size](#)). As the user space buffer is filled from the kernel buffer, there is no benefit making the user space buffer larger than the kernel buffer. The default kernel buffer size for each request is (16 * num_lines).

4.10.3.6 gpiod_edge_event_copy()

```
struct gpiod_edge_event * gpiod_edge_event_copy (
    struct gpiod_edge_event * event )
```

Copy the edge event object.

Parameters

<i>event</i>	Edge event to copy.
--------------	---------------------

Returns

Copy of the edge event or NULL on error. The returned object must be freed by the caller using [gpiod_edge_event_free](#).

4.10.3.7 gpiod_edge_event_free()

```
void gpiod_edge_event_free (
    struct gpiod_edge_event * event )
```

Free the edge event object.

Parameters

<i>event</i>	Edge event object to free.
--------------	----------------------------

4.10.3.8 gpiod_edge_event_get_event_type()

```
enum gpiod_edge_event_type gpiod_edge_event_get_event_type (
    struct gpiod_edge_event * event )
```


Get the event type.

Parameters

<i>event</i>	GPIO edge event.
--------------	------------------

Returns

The event type ([GPIOD_EDGE_EVENT_RISING_EDGE](#) or [GPIOD_EDGE_EVENT_FALLING_EDGE](#)).

4.10.3.9 `gpiod_edge_event_get_global_seqno()`

```
unsigned long gpiod_edge_event_get_global_seqno (  
    struct gpiod\_edge\_event * event )
```

Get the global sequence number of the event.

Parameters

<i>event</i>	GPIO edge event.
--------------	------------------

Returns

Sequence number of the event in the series of events for all lines in the associated line request.

4.10.3.10 `gpiod_edge_event_get_line_offset()`

```
unsigned int gpiod_edge_event_get_line_offset (  
    struct gpiod\_edge\_event * event )
```

Get the offset of the line which triggered the event.

Parameters

<i>event</i>	GPIO edge event.
--------------	------------------

Returns

Line offset.

4.10.3.11 `gpiod_edge_event_get_line_seqno()`

```
unsigned long gpiod_edge_event_get_line_seqno (  
    struct gpiod\_edge\_event * event )
```

Get the event sequence number specific to the line.

Parameters

<i>event</i>	GPIO edge event.
--------------	------------------

Returns

Sequence number of the event in the series of events only for this line within the lifetime of the associated line request.

4.10.3.12 `gpiod_edge_event_get_timestamp_ns()`

```
uint64_t gpiod_edge_event_get_timestamp_ns (
    struct gpiod_edge_event * event )
```

Get the timestamp of the event.

Parameters

<i>event</i>	GPIO edge event.
--------------	------------------

Returns

Timestamp in nanoseconds.

Note

The source clock for the timestamp depends on the `event_clock` setting for the line.

4.11 Stuff that didn't fit anywhere else**Functions**

- bool `gpiod_is_gpiochip_device` (const char *path)
Check if the file pointed to by path is a GPIO chip character device.
- const char * `gpiod_api_version` (void)
Get the API version of the library as a human-readable string.

4.11.1 Detailed Description

Various libgpiod-related functions.

4.11.2 Function Documentation

4.11.2.1 `gpiod_api_version()`

```
const char * gpiod_api_version (
    void )
```

Get the API version of the library as a human-readable string.

Returns

A valid pointer to a human-readable string containing the library version. The pointer is valid for the lifetime of the program and must not be freed by the caller.

4.11.2.2 `gpiod_is_gpiochip_device()`

```
bool gpiod_is_gpiochip_device (
    const char * path )
```

Check if the file pointed to by *path* is a GPIO chip character device.

Parameters

<i>path</i>	Path to check.
-------------	----------------

Returns

True if the file exists and is either a GPIO chip character device or a symbolic link to one.

Chapter 5

Class Documentation

5.1 `gpiod_chip` Struct Reference

```
#include <gpiod.h>
```

Public Attributes

- `int fd`
- `char * path`

5.1.1 Detailed Description

Refer to [GPIO chips](#) for functions that operate on `gpiod_chip`.

The documentation for this struct was generated from the following file:

- `src/lib/chip.c`

5.2 `gpiod_chip_info` Struct Reference

```
#include <gpiod.h>
```

Public Attributes

- `size_t num_lines`
- `char name [GPIO_MAX_NAME_SIZE+1]`
- `char label [GPIO_MAX_NAME_SIZE+1]`

5.2.1 Detailed Description

Refer to [Chip info](#) for functions that operate on `gpiod_chip_info`.

The documentation for this struct was generated from the following file:

- `src/lib/chip-info.c`

5.3 `gpiod_edge_event` Struct Reference

```
#include <gpiod.h>
```

Public Attributes

- enum [gpiod_edge_event_type](#) `event_type`
- `uint64_t` `timestamp`
- unsigned int `line_offset`
- unsigned long `global_seqno`
- unsigned long `line_seqno`

5.3.1 Detailed Description

Refer to [Line edge events handling](#) for functions that operate on `gpiod_edge_event`.

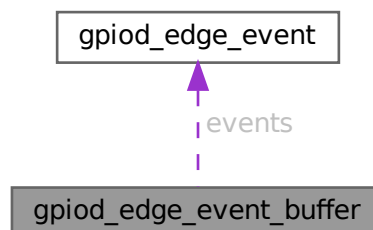
The documentation for this struct was generated from the following file:

- `src/lib/edge-event.c`

5.4 `gpiod_edge_event_buffer` Struct Reference

```
#include <gpiod.h>
```

Collaboration diagram for `gpiod_edge_event_buffer`:



Public Attributes

- `size_t` **capacity**
- `size_t` **num_events**
- `struct gpiod_edge_event *` **events**
- `struct gpio_v2_line_event *` **event_data**

5.4.1 Detailed Description

Refer to [Line edge events handling](#) for functions that operate on `gpiod_edge_event_buffer`.

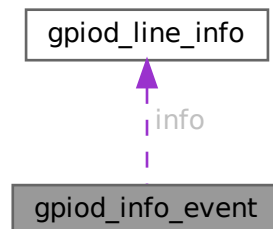
The documentation for this struct was generated from the following file:

- `src/lib/edge-event.c`

5.5 gpiod_info_event Struct Reference

```
#include <gpiod.h>
```

Collaboration diagram for `gpiod_info_event`:

**Public Attributes**

- `enum gpiod_info_event_type` **event_type**
- `uint64_t` **timestamp**
- `struct gpiod_line_info *` **info**

5.5.1 Detailed Description

Refer to [Line status watch events](#) for functions that operate on `gpiod_info_event`.

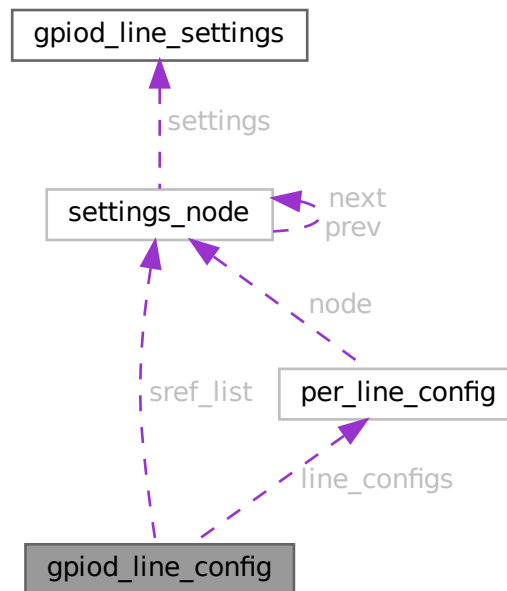
The documentation for this struct was generated from the following file:

- `src/lib/info-event.c`

5.6 gpiod_line_config Struct Reference

```
#include <gpiod.h>
```

Collaboration diagram for gpiod_line_config:



Public Attributes

- struct [per_line_config](#) `line_configs` [LINES_MAX]
- size_t `num_configs`
- enum [gpiod_line_value](#) `output_values` [LINES_MAX]
- size_t `num_output_values`
- struct [settings_node](#) * `sref_list`

5.6.1 Detailed Description

Refer to [Line configuration objects](#) for functions that operate on `gpiod_line_config`.

The documentation for this struct was generated from the following file:

- `src/lib/line-config.c`

5.7 gpiod_line_info Struct Reference

```
#include <gpiod.h>
```


Public Attributes

- unsigned int **offset**
- char **name** [GPIO_MAX_NAME_SIZE+1]
- bool **used**
- char **consumer** [GPIO_MAX_NAME_SIZE+1]
- enum [gpiod_line_direction](#) **direction**
- bool **active_low**
- enum [gpiod_line_bias](#) **bias**
- enum [gpiod_line_drive](#) **drive**
- enum [gpiod_line_edge](#) **edge**
- enum [gpiod_line_clock](#) **event_clock**
- bool **debounced**
- unsigned long **debounce_period_us**

5.7.1 Detailed Description

Refer to [Line info](#) for functions that operate on [gpiod_line_info](#).

The documentation for this struct was generated from the following file:

- `src/lib/line-info.c`

5.8 gpiod_line_request Struct Reference

```
#include <gpiod.h>
```

Public Attributes

- char * **chip_name**
- unsigned int **offsets** [GPIO_V2_LINES_MAX]
- size_t **num_lines**
- int **fd**

5.8.1 Detailed Description

Refer to [Line request operations](#) for functions that operate on [gpiod_line_request](#).

The documentation for this struct was generated from the following file:

- `src/lib/line-request.c`

5.9 gpiod_line_settings Struct Reference

```
#include <gpiod.h>
```

Public Attributes

- enum [gpiod_line_direction](#) **direction**
- enum [gpiod_line_edge](#) **edge_detection**
- enum [gpiod_line_drive](#) **drive**
- enum [gpiod_line_bias](#) **bias**
- bool **active_low**
- enum [gpiod_line_clock](#) **event_clock**
- unsigned long **debounce_period_us**
- enum [gpiod_line_value](#) **output_value**

5.9.1 Detailed Description

Refer to [Line settings objects](#) for functions that operate on [gpiod_line_settings](#).

The documentation for this struct was generated from the following file:

- `src/lib/line-settings.c`

5.10 gpiod_request_config Struct Reference

```
#include <gpiod.h>
```

Public Attributes

- char **consumer** [GPIO_MAX_NAME_SIZE]
- size_t **event_buffer_size**

5.10.1 Detailed Description

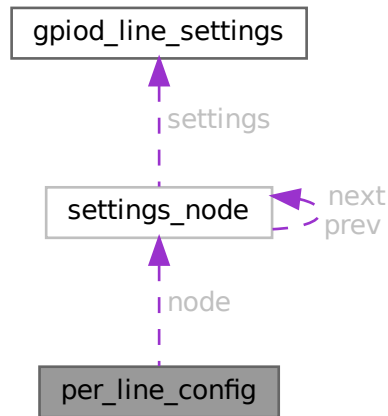
Refer to [Request configuration objects](#) for functions that operate on [gpiod_request_config](#).

The documentation for this struct was generated from the following file:

- `src/lib/request-config.c`

5.11 per_line_config Struct Reference

Collaboration diagram for per_line_config:



Public Attributes

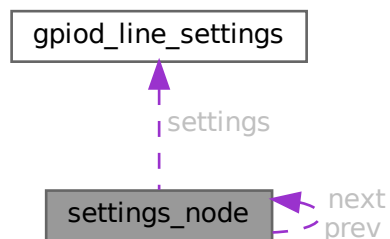
- unsigned int **offset**
- struct `settings_node` * **node**

The documentation for this struct was generated from the following file:

- `src/lib/line-config.c`

5.12 settings_node Struct Reference

Collaboration diagram for settings_node:



Public Attributes

- struct [settings_node](#) * **prev**
- struct [settings_node](#) * **next**
- struct [gpiod_line_settings](#) * **settings**
- unsigned int **refcnt**

The documentation for this struct was generated from the following file:

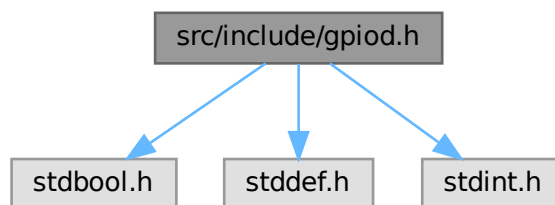
- `src/lib/line-config.c`

Chapter 6

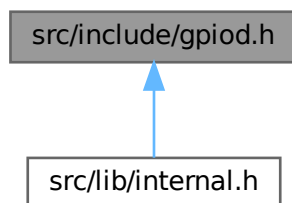
File Documentation

6.1 src/include/gpiod.h File Reference

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
Include dependency graph for gpiod.h:
```



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `gpiod_line_value` { `GPIOD_LINE_VALUE_ERROR` = -1 , `GPIOD_LINE_VALUE_INACTIVE` = 0 , `GPIOD_LINE_VALUE_ACTIVE` = 1 }
Logical line state.
- enum `gpiod_line_direction` { `GPIOD_LINE_DIRECTION_AS_IS` = 1 , `GPIOD_LINE_DIRECTION_INPUT` , `GPIOD_LINE_DIRECTION_OUTPUT` }
Direction settings.
- enum `gpiod_line_edge` { `GPIOD_LINE_EDGE_NONE` = 1 , `GPIOD_LINE_EDGE_RISING` , `GPIOD_LINE_EDGE_FALLING` , `GPIOD_LINE_EDGE_BOTH` }
Edge detection settings.
- enum `gpiod_line_bias` { `GPIOD_LINE_BIAS_AS_IS` = 1 , `GPIOD_LINE_BIAS_UNKNOWN` , `GPIOD_LINE_BIAS_DISABLED` , `GPIOD_LINE_BIAS_PULL_UP` , `GPIOD_LINE_BIAS_PULL_DOWN` }
Internal bias settings.
- enum `gpiod_line_drive` { `GPIOD_LINE_DRIVE_PUSH_PULL` = 1 , `GPIOD_LINE_DRIVE_OPEN_DRAIN` , `GPIOD_LINE_DRIVE_OPEN_SOURCE` }
Drive settings.
- enum `gpiod_line_clock` { `GPIOD_LINE_CLOCK_MONOTONIC` = 1 , `GPIOD_LINE_CLOCK_REALTIME` , `GPIOD_LINE_CLOCK_HTE` }
Clock settings.
- enum `gpiod_info_event_type` { `GPIOD_INFO_EVENT_LINE_REQUESTED` = 1 , `GPIOD_INFO_EVENT_LINE_RELEASED` , `GPIOD_INFO_EVENT_LINE_CONFIG_CHANGED` }
Line status change event types.
- enum `gpiod_edge_event_type` { `GPIOD_EDGE_EVENT_RISING_EDGE` = 1 , `GPIOD_EDGE_EVENT_FALLING_EDGE` }
Event types.

Functions

- struct `gpiod_chip` * `gpiod_chip_open` (const char *path)
Open a chip by path.
- void `gpiod_chip_close` (struct `gpiod_chip` *chip)
Close the chip and release all associated resources.
- struct `gpiod_chip_info` * `gpiod_chip_get_info` (struct `gpiod_chip` *chip)
Get information about the chip.
- const char * `gpiod_chip_get_path` (struct `gpiod_chip` *chip)
Get the path used to open the chip.
- struct `gpiod_line_info` * `gpiod_chip_get_line_info` (struct `gpiod_chip` *chip, unsigned int offset)
Get a snapshot of information about a line.
- struct `gpiod_line_info` * `gpiod_chip_watch_line_info` (struct `gpiod_chip` *chip, unsigned int offset)
Get a snapshot of the status of a line and start watching it for future changes.
- int `gpiod_chip_unwatch_line_info` (struct `gpiod_chip` *chip, unsigned int offset)
Stop watching a line for status changes.
- int `gpiod_chip_get_fd` (struct `gpiod_chip` *chip)
Get the file descriptor associated with the chip.
- int `gpiod_chip_wait_info_event` (struct `gpiod_chip` *chip, int64_t timeout_ns)
Wait for line status change events on any of the watched lines on the chip.
- struct `gpiod_info_event` * `gpiod_chip_read_info_event` (struct `gpiod_chip` *chip)
Read a single line status change event from the chip.
- int `gpiod_chip_get_line_offset_from_name` (struct `gpiod_chip` *chip, const char *name)

- Map a line's name to its offset within the chip.*
- struct [gpiod_line_request](#) * [gpiod_chip_request_lines](#) (struct [gpiod_chip](#) *chip, struct [gpiod_request_config](#) *req_cfg, struct [gpiod_line_config](#) *line_cfg)
- Request a set of lines for exclusive usage.*
- void [gpiod_chip_info_free](#) (struct [gpiod_chip_info](#) *info)
- Free a chip info object and release all associated resources.*
- const char * [gpiod_chip_info_get_name](#) (struct [gpiod_chip_info](#) *info)
- Get the name of the chip as represented in the kernel.*
- const char * [gpiod_chip_info_get_label](#) (struct [gpiod_chip_info](#) *info)
- Get the label of the chip as represented in the kernel.*
- size_t [gpiod_chip_info_get_num_lines](#) (struct [gpiod_chip_info](#) *info)
- Get the number of lines exposed by the chip.*
- void [gpiod_line_info_free](#) (struct [gpiod_line_info](#) *info)
- Free a line info object and release all associated resources.*
- struct [gpiod_line_info](#) * [gpiod_line_info_copy](#) (struct [gpiod_line_info](#) *info)
- Copy a line info object.*
- unsigned int [gpiod_line_info_get_offset](#) (struct [gpiod_line_info](#) *info)
- Get the offset of the line.*
- const char * [gpiod_line_info_get_name](#) (struct [gpiod_line_info](#) *info)
- Get the name of the line.*
- bool [gpiod_line_info_is_used](#) (struct [gpiod_line_info](#) *info)
- Check if the line is in use.*
- const char * [gpiod_line_info_get_consumer](#) (struct [gpiod_line_info](#) *info)
- Get the name of the consumer of the line.*
- enum [gpiod_line_direction](#) [gpiod_line_info_get_direction](#) (struct [gpiod_line_info](#) *info)
- Get the direction setting of the line.*
- enum [gpiod_line_edge](#) [gpiod_line_info_get_edge_detection](#) (struct [gpiod_line_info](#) *info)
- Get the edge detection setting of the line.*
- enum [gpiod_line_bias](#) [gpiod_line_info_get_bias](#) (struct [gpiod_line_info](#) *info)
- Get the bias setting of the line.*
- enum [gpiod_line_drive](#) [gpiod_line_info_get_drive](#) (struct [gpiod_line_info](#) *info)
- Get the drive setting of the line.*
- bool [gpiod_line_info_is_active_low](#) (struct [gpiod_line_info](#) *info)
- Check if the logical value of the line is inverted compared to the physical.*
- bool [gpiod_line_info_is_debounced](#) (struct [gpiod_line_info](#) *info)
- Check if the line is debounced (either by hardware or by the kernel software debouncer).*
- unsigned long [gpiod_line_info_get_debounce_period_us](#) (struct [gpiod_line_info](#) *info)
- Get the debounce period of the line, in microseconds.*
- enum [gpiod_line_clock](#) [gpiod_line_info_get_event_clock](#) (struct [gpiod_line_info](#) *info)
- Get the event clock setting used for edge event timestamps for the line.*
- void [gpiod_info_event_free](#) (struct [gpiod_info_event](#) *event)
- Free the info event object and release all associated resources.*
- enum [gpiod_info_event_type](#) [gpiod_info_event_get_event_type](#) (struct [gpiod_info_event](#) *event)
- Get the event type of the status change event.*
- uint64_t [gpiod_info_event_get_timestamp_ns](#) (struct [gpiod_info_event](#) *event)
- Get the timestamp of the event.*
- struct [gpiod_line_info](#) * [gpiod_info_event_get_line_info](#) (struct [gpiod_info_event](#) *event)
- Get the snapshot of line-info associated with the event.*
- struct [gpiod_line_settings](#) * [gpiod_line_settings_new](#) (void)
- Create a new line settings object.*
- void [gpiod_line_settings_free](#) (struct [gpiod_line_settings](#) *settings)

- Free the line settings object and release all associated resources.*

 - void [gpiod_line_settings_reset](#) (struct [gpiod_line_settings](#) *settings)
- Reset the line settings object to its default values.*

 - struct [gpiod_line_settings](#) * [gpiod_line_settings_copy](#) (struct [gpiod_line_settings](#) *settings)
- Copy the line settings object.*

 - int [gpiod_line_settings_set_direction](#) (struct [gpiod_line_settings](#) *settings, enum [gpiod_line_direction](#) direction)
- Set direction.*

 - enum [gpiod_line_direction](#) [gpiod_line_settings_get_direction](#) (struct [gpiod_line_settings](#) *settings)
- Get direction.*

 - int [gpiod_line_settings_set_edge_detection](#) (struct [gpiod_line_settings](#) *settings, enum [gpiod_line_edge](#) edge)
- Set edge detection.*

 - enum [gpiod_line_edge](#) [gpiod_line_settings_get_edge_detection](#) (struct [gpiod_line_settings](#) *settings)
- Get edge detection.*

 - int [gpiod_line_settings_set_bias](#) (struct [gpiod_line_settings](#) *settings, enum [gpiod_line_bias](#) bias)
- Set bias.*

 - enum [gpiod_line_bias](#) [gpiod_line_settings_get_bias](#) (struct [gpiod_line_settings](#) *settings)
- Get bias.*

 - int [gpiod_line_settings_set_drive](#) (struct [gpiod_line_settings](#) *settings, enum [gpiod_line_drive](#) drive)
- Set drive.*

 - enum [gpiod_line_drive](#) [gpiod_line_settings_get_drive](#) (struct [gpiod_line_settings](#) *settings)
- Get drive.*

 - void [gpiod_line_settings_set_active_low](#) (struct [gpiod_line_settings](#) *settings, bool active_low)
- Set active-low setting.*

 - bool [gpiod_line_settings_get_active_low](#) (struct [gpiod_line_settings](#) *settings)
- Get active-low setting.*

 - void [gpiod_line_settings_set_debounce_period_us](#) (struct [gpiod_line_settings](#) *settings, unsigned long period)
- Set debounce period.*

 - unsigned long [gpiod_line_settings_get_debounce_period_us](#) (struct [gpiod_line_settings](#) *settings)
- Get debounce period.*

 - int [gpiod_line_settings_set_event_clock](#) (struct [gpiod_line_settings](#) *settings, enum [gpiod_line_clock](#) event↔_clock)
- Set event clock.*

 - enum [gpiod_line_clock](#) [gpiod_line_settings_get_event_clock](#) (struct [gpiod_line_settings](#) *settings)
- Get event clock setting.*

 - int [gpiod_line_settings_set_output_value](#) (struct [gpiod_line_settings](#) *settings, enum [gpiod_line_value](#) value)
- Set the output value.*

 - enum [gpiod_line_value](#) [gpiod_line_settings_get_output_value](#) (struct [gpiod_line_settings](#) *settings)
- Get the output value.*

 - struct [gpiod_line_config](#) * [gpiod_line_config_new](#) (void)
- Create a new line config object.*

 - void [gpiod_line_config_free](#) (struct [gpiod_line_config](#) *config)
- Free the line config object and release all associated resources.*

 - void [gpiod_line_config_reset](#) (struct [gpiod_line_config](#) *config)
- Reset the line config object.*

 - int [gpiod_line_config_add_line_settings](#) (struct [gpiod_line_config](#) *config, const unsigned int *offsets, size_t num_offsets, struct [gpiod_line_settings](#) *settings)
- Add line settings for a set of offsets.*

- struct [gpiod_line_settings](#) * [gpiod_line_config_get_line_settings](#) (struct [gpiod_line_config](#) *config, unsigned int offset)
Get line settings for offset.
- int [gpiod_line_config_set_output_values](#) (struct [gpiod_line_config](#) *config, const enum [gpiod_line_value](#) *values, size_t num_values)
Set output values for a number of lines.
- size_t [gpiod_line_config_get_num_configured_offsets](#) (struct [gpiod_line_config](#) *config)
Get the number of configured line offsets.
- size_t [gpiod_line_config_get_configured_offsets](#) (struct [gpiod_line_config](#) *config, unsigned int *offsets, size_t max_offsets)
Get configured offsets.
- struct [gpiod_request_config](#) * [gpiod_request_config_new](#) (void)
Create a new request config object.
- void [gpiod_request_config_free](#) (struct [gpiod_request_config](#) *config)
Free the request config object and release all associated resources.
- void [gpiod_request_config_set_consumer](#) (struct [gpiod_request_config](#) *config, const char *consumer)
Set the consumer name for the request.
- const char * [gpiod_request_config_get_consumer](#) (struct [gpiod_request_config](#) *config)
Get the consumer name configured in the request config.
- void [gpiod_request_config_set_event_buffer_size](#) (struct [gpiod_request_config](#) *config, size_t event_buffer_size)
Set the size of the kernel event buffer for the request.
- size_t [gpiod_request_config_get_event_buffer_size](#) (struct [gpiod_request_config](#) *config)
Get the edge event buffer size for the request config.
- void [gpiod_line_request_release](#) (struct [gpiod_line_request](#) *request)
Release the requested lines and free all associated resources.
- const char * [gpiod_line_request_get_chip_name](#) (struct [gpiod_line_request](#) *request)
Get the name of the chip this request was made on.
- size_t [gpiod_line_request_get_num_requested_lines](#) (struct [gpiod_line_request](#) *request)
Get the number of lines in the request.
- size_t [gpiod_line_request_get_requested_offsets](#) (struct [gpiod_line_request](#) *request, unsigned int *offsets, size_t max_offsets)
Get the offsets of the lines in the request.
- enum [gpiod_line_value](#) [gpiod_line_request_get_value](#) (struct [gpiod_line_request](#) *request, unsigned int offset)
Get the value of a single requested line.
- int [gpiod_line_request_get_values_subset](#) (struct [gpiod_line_request](#) *request, size_t num_values, const unsigned int *offsets, enum [gpiod_line_value](#) *values)
Get the values of a subset of requested lines.
- int [gpiod_line_request_get_values](#) (struct [gpiod_line_request](#) *request, enum [gpiod_line_value](#) *values)
Get the values of all requested lines.
- int [gpiod_line_request_set_value](#) (struct [gpiod_line_request](#) *request, unsigned int offset, enum [gpiod_line_value](#) value)
Set the value of a single requested line.
- int [gpiod_line_request_set_values_subset](#) (struct [gpiod_line_request](#) *request, size_t num_values, const unsigned int *offsets, const enum [gpiod_line_value](#) *values)
Set the values of a subset of requested lines.
- int [gpiod_line_request_set_values](#) (struct [gpiod_line_request](#) *request, const enum [gpiod_line_value](#) *values)
Set the values of all lines associated with a request.
- int [gpiod_line_request_reconfigure_lines](#) (struct [gpiod_line_request](#) *request, struct [gpiod_line_config](#) *config)

- Update the configuration of lines associated with a line request.*
- int `gpiod_line_request_get_fd` (struct `gpiod_line_request` *request)
Get the file descriptor associated with a line request.
- int `gpiod_line_request_wait_edge_events` (struct `gpiod_line_request` *request, int64_t timeout_ns)
Wait for edge events on any of the requested lines.
- int `gpiod_line_request_read_edge_events` (struct `gpiod_line_request` *request, struct `gpiod_edge_event_buffer` *buffer, size_t max_events)
Read a number of edge events from a line request.
- void `gpiod_edge_event_free` (struct `gpiod_edge_event` *event)
Free the edge event object.
- struct `gpiod_edge_event` * `gpiod_edge_event_copy` (struct `gpiod_edge_event` *event)
Copy the edge event object.
- enum `gpiod_edge_event_type` `gpiod_edge_event_get_event_type` (struct `gpiod_edge_event` *event)
Get the event type.
- uint64_t `gpiod_edge_event_get_timestamp_ns` (struct `gpiod_edge_event` *event)
Get the timestamp of the event.
- unsigned int `gpiod_edge_event_get_line_offset` (struct `gpiod_edge_event` *event)
Get the offset of the line which triggered the event.
- unsigned long `gpiod_edge_event_get_global_seqno` (struct `gpiod_edge_event` *event)
Get the global sequence number of the event.
- unsigned long `gpiod_edge_event_get_line_seqno` (struct `gpiod_edge_event` *event)
Get the event sequence number specific to the line.
- struct `gpiod_edge_event_buffer` * `gpiod_edge_event_buffer_new` (size_t capacity)
Create a new edge event buffer.
- size_t `gpiod_edge_event_buffer_get_capacity` (struct `gpiod_edge_event_buffer` *buffer)
Get the capacity (the max number of events that can be stored) of the event buffer.
- void `gpiod_edge_event_buffer_free` (struct `gpiod_edge_event_buffer` *buffer)
Free the edge event buffer and release all associated resources.
- struct `gpiod_edge_event` * `gpiod_edge_event_buffer_get_event` (struct `gpiod_edge_event_buffer` *buffer, unsigned long index)
Get an event stored in the buffer.
- size_t `gpiod_edge_event_buffer_get_num_events` (struct `gpiod_edge_event_buffer` *buffer)
Get the number of events a buffer has stored.
- bool `gpiod_is_gpiochip_device` (const char *path)
Check if the file pointed to by path is a GPIO chip character device.
- const char * `gpiod_api_version` (void)
Get the API version of the library as a human-readable string.

6.2 gpiod.h

[Go to the documentation of this file.](#)

```

00001 /* SPDX-License-Identifier: LGPL-2.1-or-later */
00002 /* SPDX-FileCopyrightText: 2017-2022 Bartosz Golaszewski <brgl@bgdev.pl> */
00003
00008 #ifndef __LIBGPIOD_GPIOD_H__
00009 #define __LIBGPIOD_GPIOD_H__
00010
00011 #include <stdbool.h>
00012 #include <stddef.h>
00013 #include <stdint.h>
00014
00015 #ifdef __cplusplus
00016 extern "C" {
00017 #endif
00018
```

```

00027 struct gpiod_chip;
00028
00037 struct gpiod_chip_info;
00038
00047 struct gpiod_line_info;
00048
00058 struct gpiod_line_settings;
00059
00068 struct gpiod_line_config;
00069
00079 struct gpiod_request_config;
00080
00090 struct gpiod_line_request;
00091
00100 struct gpiod_info_event;
00101
00110 struct gpiod_edge_event;
00111
00121 struct gpiod_edge_event_buffer;
00122
00141 struct gpiod_chip *gpiod_chip_open(const char *path);
00142
00147 void gpiod_chip_close(struct gpiod_chip *chip);
00148
00155 struct gpiod_chip_info *gpiod_chip_get_info(struct gpiod_chip *chip);
00156
00164 const char *gpiod_chip_get_path(struct gpiod_chip *chip);
00165
00173 struct gpiod_line_info *gpiod_chip_get_line_info(struct gpiod_chip *chip,
00174 unsigned int offset);
00175
00186 struct gpiod_line_info *gpiod_chip_watch_line_info(struct gpiod_chip *chip,
00187 unsigned int offset);
00188
00195 int gpiod_chip_unwatch_line_info(struct gpiod_chip *chip, unsigned int offset);
00196
00206 int gpiod_chip_get_fd(struct gpiod_chip *chip);
00207
00219 int gpiod_chip_wait_info_event(struct gpiod_chip *chip, int64_t timeout_ns);
00220
00228 struct gpiod_info_event *gpiod_chip_read_info_event(struct gpiod_chip *chip);
00229
00238 int gpiod_chip_get_line_offset_from_name(struct gpiod_chip *chip,
00239 const char *name);
00240
00249 struct gpiod_line_request *
00250 gpiod_chip_request_lines(struct gpiod_chip *chip,
00251 struct gpiod_request_config *req_cfg,
00252 struct gpiod_line_config *line_cfg);
00253
00278 void gpiod_chip_info_free(struct gpiod_chip_info *info);
00279
00287 const char *gpiod_chip_info_get_name(struct gpiod_chip_info *info);
00288
00296 const char *gpiod_chip_info_get_label(struct gpiod_chip_info *info);
00297
00303 size_t gpiod_chip_info_get_num_lines(struct gpiod_chip_info *info);
00304
00317 enum gpiod_line_value {
00318     GPIOD_LINE_VALUE_ERROR = -1,
00320     GPIOD_LINE_VALUE_INACTIVE = 0,
00322     GPIOD_LINE_VALUE_ACTIVE = 1,
00324 };
00325
00329 enum gpiod_line_direction {
00330     GPIOD_LINE_DIRECTION_AS_IS = 1,
00332     GPIOD_LINE_DIRECTION_INPUT,
00335     GPIOD_LINE_DIRECTION_OUTPUT,
00337 };
00338
00342 enum gpiod_line_edge {
00343     GPIOD_LINE_EDGE_NONE = 1,
00345     GPIOD_LINE_EDGE_RISING,
00347     GPIOD_LINE_EDGE_FALLING,
00349     GPIOD_LINE_EDGE_BOTH,
00351 };
00352
00356 enum gpiod_line_bias {
00357     GPIOD_LINE_BIAS_AS_IS = 1,
00359     GPIOD_LINE_BIAS_UNKNOWN,
00361     GPIOD_LINE_BIAS_DISABLED,
00363     GPIOD_LINE_BIAS_PULL_UP,
00365     GPIOD_LINE_BIAS_PULL_DOWN,
00367 };
00368
00372 enum gpiod_line_drive {
00373     GPIOD_LINE_DRIVE_PUSH_PULL = 1,

```

```
00375     GPIOD_LINE_DRIVE_OPEN_DRAIN,
00377     GPIOD_LINE_DRIVE_OPEN_SOURCE,
00379 };
00380
00384 enum gpiod_line_clock {
00385     GPIOD_LINE_CLOCK_MONOTONIC = 1,
00387     GPIOD_LINE_CLOCK_REALTIME,
00389     GPIOD_LINE_CLOCK_HTE,
00391 };
00392
00419 void gpiod_line_info_free(struct gpiod_line_info *info);
00420
00427 struct gpiod_line_info *gpiod_line_info_copy(struct gpiod_line_info *info);
00428
00437 unsigned int gpiod_line_info_get_offset(struct gpiod_line_info *info);
00438
00447 const char *gpiod_line_info_get_name(struct gpiod_line_info *info);
00448
00459 bool gpiod_line_info_is_used(struct gpiod_line_info *info);
00460
00470 const char *gpiod_line_info_get_consumer(struct gpiod_line_info *info);
00471
00478 enum gpiod_line_direction
00479 gpiod_line_info_get_direction(struct gpiod_line_info *info);
00480
00487 enum gpiod_line_edge
00488 gpiod_line_info_get_edge_detection(struct gpiod_line_info *info);
00489
00496 enum gpiod_line_bias
00497 gpiod_line_info_get_bias(struct gpiod_line_info *info);
00498
00505 enum gpiod_line_drive
00506 gpiod_line_info_get_drive(struct gpiod_line_info *info);
00507
00514 bool gpiod_line_info_is_active_low(struct gpiod_line_info *info);
00515
00522 bool gpiod_line_info_is_debounced(struct gpiod_line_info *info);
00523
00530 unsigned long
00531 gpiod_line_info_get_debounce_period_us(struct gpiod_line_info *info);
00532
00540 enum gpiod_line_clock
00541 gpiod_line_info_get_event_clock(struct gpiod_line_info *info);
00542
00561 enum gpiod_info_event_type {
00562     GPIOD_INFO_EVENT_LINE_REQUESTED = 1,
00564     GPIOD_INFO_EVENT_LINE_RELEASED,
00566     GPIOD_INFO_EVENT_LINE_CONFIG_CHANGED,
00568 };
00569
00574 void gpiod_info_event_free(struct gpiod_info_event *event);
00575
00583 enum gpiod_info_event_type
00584 gpiod_info_event_get_event_type(struct gpiod_info_event *event);
00585
00591 uint64_t gpiod_info_event_get_timestamp_ns(struct gpiod_info_event *event);
00592
00606 struct gpiod_line_info *
00607 gpiod_info_event_get_line_info(struct gpiod_info_event *event);
00608
00631 struct gpiod_line_settings *gpiod_line_settings_new(void);
00632
00637 void gpiod_line_settings_free(struct gpiod_line_settings *settings);
00638
00643 void gpiod_line_settings_reset(struct gpiod_line_settings *settings);
00644
00651 struct gpiod_line_settings *
00652 gpiod_line_settings_copy(struct gpiod_line_settings *settings);
00653
00660 int gpiod_line_settings_set_direction(struct gpiod_line_settings *settings,
00661                                     enum gpiod_line_direction direction);
00662
00668 enum gpiod_line_direction
00669 gpiod_line_settings_get_direction(struct gpiod_line_settings *settings);
00670
00677 int gpiod_line_settings_set_edge_detection(struct gpiod_line_settings *settings,
00678                                           enum gpiod_line_edge edge);
00679
00685 enum gpiod_line_edge
00686 gpiod_line_settings_get_edge_detection(struct gpiod_line_settings *settings);
00687
00694 int gpiod_line_settings_set_bias(struct gpiod_line_settings *settings,
00695                                 enum gpiod_line_bias bias);
00696
00702 enum gpiod_line_bias
00703 gpiod_line_settings_get_bias(struct gpiod_line_settings *settings);
00704
```

```
00711 int gpiod_line_settings_set_drive(struct gpiod_line_settings *settings,
00712     enum gpiod_line_drive drive);
00713
00719 enum gpiod_line_drive
00720 gpiod_line_settings_get_drive(struct gpiod_line_settings *settings);
00721
00727 void gpiod_line_settings_set_active_low(struct gpiod_line_settings *settings,
00728     bool active_low);
00729
00735 bool gpiod_line_settings_get_active_low(struct gpiod_line_settings *settings);
00736
00742 void
00743 gpiod_line_settings_set_debounce_period_us(struct gpiod_line_settings *settings,
00744     unsigned long period);
00745
00751 unsigned long
00752 gpiod_line_settings_get_debounce_period_us(
00753     struct gpiod_line_settings *settings);
00754
00761 int gpiod_line_settings_set_event_clock(struct gpiod_line_settings *settings,
00762     enum gpiod_line_clock event_clock);
00763
00769 enum gpiod_line_clock
00770 gpiod_line_settings_get_event_clock(struct gpiod_line_settings *settings);
00771
00778 int gpiod_line_settings_set_output_value(struct gpiod_line_settings *settings,
00779     enum gpiod_line_value value);
00780
00786 enum gpiod_line_value
00787 gpiod_line_settings_get_output_value(struct gpiod_line_settings *settings);
00788
00817 struct gpiod_line_config *gpiod_line_config_new(void);
00818
00823 void gpiod_line_config_free(struct gpiod_line_config *config);
00824
00832 void gpiod_line_config_reset(struct gpiod_line_config *config);
00833
00842 int gpiod_line_config_add_line_settings(struct gpiod_line_config *config,
00843     const unsigned int *offsets,
00844     size_t num_offsets,
00845     struct gpiod_line_settings *settings);
00846
00854 struct gpiod_line_settings *
00855 gpiod_line_config_get_line_settings(struct gpiod_line_config *config,
00856     unsigned int offset);
00857
00879 int gpiod_line_config_set_output_values(struct gpiod_line_config *config,
00880     const enum gpiod_line_value *values,
00881     size_t num_values);
00882
00888 size_t
00889 gpiod_line_config_get_num_configured_offsets(struct gpiod_line_config *config);
00890
00902 size_t
00903 gpiod_line_config_get_configured_offsets(struct gpiod_line_config *config,
00904     unsigned int *offsets,
00905     size_t max_offsets);
00906
00926 struct gpiod_request_config *gpiod_request_config_new(void);
00927
00932 void gpiod_request_config_free(struct gpiod_request_config *config);
00933
00941 void gpiod_request_config_set_consumer(struct gpiod_request_config *config,
00942     const char *consumer);
00943
00949 const char *
00950 gpiod_request_config_get_consumer(struct gpiod_request_config *config);
00951
00961 void
00962 gpiod_request_config_set_event_buffer_size(struct gpiod_request_config *config,
00963     size_t event_buffer_size);
00964
00970 size_t
00971 gpiod_request_config_get_event_buffer_size(struct gpiod_request_config *config);
00972
00986 void gpiod_line_request_release(struct gpiod_line_request *request);
00987
00994 const char *
00995 gpiod_line_request_get_chip_name(struct gpiod_line_request *request);
00996
01002 size_t
01003 gpiod_line_request_get_num_requested_lines(struct gpiod_line_request *request);
01004
01016 size_t
01017 gpiod_line_request_get_requested_offsets(struct gpiod_line_request *request,
01018     unsigned int *offsets,
01019     size_t max_offsets);
```

```

01020
01027 enum gpiod_line_value
01028 gpiod_line_request_get_value(struct gpiod_line_request *request,
01029                             unsigned int offset);
01030
01042 int gpiod_line_request_get_values_subset(struct gpiod_line_request *request,
01043                                         size_t num_values,
01044                                         const unsigned int *offsets,
01045                                         enum gpiod_line_value *values);
01046
01058 int gpiod_line_request_get_values(struct gpiod_line_request *request,
01059                                   enum gpiod_line_value *values);
01060
01068 int gpiod_line_request_set_value(struct gpiod_line_request *request,
01069                                  unsigned int offset,
01070                                  enum gpiod_line_value value);
01071
01084 int gpiod_line_request_set_values_subset(struct gpiod_line_request *request,
01085                                           size_t num_values,
01086                                           const unsigned int *offsets,
01087                                           const enum gpiod_line_value *values);
01088
01100 int gpiod_line_request_set_values(struct gpiod_line_request *request,
01101                                   const enum gpiod_line_value *values);
01102
01114 int gpiod_line_request_reconfigure_lines(struct gpiod_line_request *request,
01115                                           struct gpiod_line_config *config);
01116
01125 int gpiod_line_request_get_fd(struct gpiod_line_request *request);
01126
01140 int gpiod_line_request_wait_edge_events(struct gpiod_line_request *request,
01141                                          int64_t timeout_ns);
01142
01154 int gpiod_line_request_read_edge_events(struct gpiod_line_request *request,
01155                                          struct gpiod_edge_event_buffer *buffer,
01156                                          size_t max_events);
01157
01179 enum gpiod_edge_event_type {
01180     GPIOD_EDGE_EVENT_RISING_EDGE = 1,
01182     GPIOD_EDGE_EVENT_FALLING_EDGE,
01184 };
01185
01190 void gpiod_edge_event_free(struct gpiod_edge_event *event);
01191
01198 struct gpiod_edge_event *gpiod_edge_event_copy(struct gpiod_edge_event *event);
01199
01206 enum gpiod_edge_event_type
01207 gpiod_edge_event_get_event_type(struct gpiod_edge_event *event);
01208
01216 uint64_t gpiod_edge_event_get_timestamp_ns(struct gpiod_edge_event *event);
01217
01223 unsigned int gpiod_edge_event_get_line_offset(struct gpiod_edge_event *event);
01224
01231 unsigned long gpiod_edge_event_get_global_seqno(struct gpiod_edge_event *event);
01232
01239 unsigned long gpiod_edge_event_get_line_seqno(struct gpiod_edge_event *event);
01240
01253 struct gpiod_edge_event_buffer *
01254 gpiod_edge_event_buffer_new(size_t capacity);
01255
01262 size_t
01263 gpiod_edge_event_buffer_get_capacity(struct gpiod_edge_event_buffer *buffer);
01264
01269 void gpiod_edge_event_buffer_free(struct gpiod_edge_event_buffer *buffer);
01270
01285 struct gpiod_edge_event *
01286 gpiod_edge_event_buffer_get_event(struct gpiod_edge_event_buffer *buffer,
01287                                   unsigned long index);
01288
01294 size_t
01295 gpiod_edge_event_buffer_get_num_events(struct gpiod_edge_event_buffer *buffer);
01296
01312 bool gpiod_is_gpiochip_device(const char *path);
01313
01320 const char *gpiod_api_version(void);
01321
01326 #ifdef __cplusplus
01327 } /* extern "C" */
01328 #endif
01329
01330 #endif /* __LIBGPIOD_GPIOD_H__ */

```

6.3 internal.h

```

00001 /* SPDX-License-Identifier: LGPL-2.1-or-later */
00002 /* SPDX-FileCopyrightText: 2021 Bartosz Golaszewski <bgolaszewski@baylibre.com> */
00003
00004 #ifndef __LIBGPIOD_GPIOD_INTERNAL_H__
00005 #define __LIBGPIOD_GPIOD_INTERNAL_H__
00006
00007 #include <gpiod.h>
00008 #include <stddef.h>
00009 #include <stdint.h>
00010
00011 #include "uapi/gpio.h"
00012
00013 /* For internal library use only. */
00014
00015 #define GPIOD_API __attribute__((visibility("default")))
00016 #define GPIOD_BIT(nr) (1UL < (nr))
00017
00018 bool gpiod_check_gpiochip_device(const char *path, bool set_errno);
00019
00020 struct gpiod_chip_info *
00021 gpiod_chip_info_from_uapi(struct gpiochip_info *uapi_info);
00022 struct gpiod_line_info *
00023 gpiod_line_info_from_uapi(struct gpio_v2_line_info *uapi_info);
00024 void gpiod_request_config_to_uapi(struct gpiod_request_config *config,
00025                                  struct gpio_v2_line_request *uapi_req);
00026 int gpiod_line_config_to_uapi(struct gpiod_line_config *config,
00027                              struct gpio_v2_line_request *uapi_cfg);
00028 struct gpiod_line_request *
00029 gpiod_line_request_from_uapi(struct gpio_v2_line_request *uapi_req,
00030                             const char *chip_name);
00031 int gpiod_edge_event_buffer_read_fd(int fd,
00032                                     struct gpiod_edge_event_buffer *buffer,
00033                                     size_t max_events);
00034 struct gpiod_info_event *
00035 gpiod_info_event_from_uapi(struct gpio_v2_line_info_changed *uapi_evt);
00036 struct gpiod_info_event *gpiod_info_event_read_fd(int fd);
00037
00038 int gpiod_poll_fd(int fd, int64_t timeout);
00039 int gpiod_set_output_value(enum gpiod_line_value in,
00040                           enum gpiod_line_value *out);
00041 int gpiod_ioctl(int fd, unsigned long request, void *arg);
00042
00043 void gpiod_line_mask_zero(uint64_t *mask);
00044 bool gpiod_line_mask_test_bit(const uint64_t *mask, unsigned int nr);
00045 void gpiod_line_mask_set_bit(uint64_t *mask, unsigned int nr);
00046 void gpiod_line_mask_assign_bit(uint64_t *mask, unsigned int nr, bool value);
00047
00048 #endif /* __LIBGPIOD_GPIOD_INTERNAL_H__ */

```


Index

Chip info, [12](#)

- [gpiod_chip_info_free](#), [13](#)
- [gpiod_chip_info_get_label](#), [13](#)
- [gpiod_chip_info_get_name](#), [13](#)
- [gpiod_chip_info_get_num_lines](#), [13](#)

GPIO chips, [7](#)

- [gpiod_chip_close](#), [8](#)
 - [gpiod_chip_get_fd](#), [8](#)
 - [gpiod_chip_get_info](#), [8](#)
 - [gpiod_chip_get_line_info](#), [8](#)
 - [gpiod_chip_get_line_offset_from_name](#), [9](#)
 - [gpiod_chip_get_path](#), [9](#)
 - [gpiod_chip_open](#), [10](#)
 - [gpiod_chip_read_info_event](#), [10](#)
 - [gpiod_chip_request_lines](#), [10](#)
 - [gpiod_chip_unwatch_line_info](#), [11](#)
 - [gpiod_chip_wait_info_event](#), [11](#)
 - [gpiod_chip_watch_line_info](#), [12](#)
- [gpiod_api_version](#)
Stuff that didn't fit anywhere else, [51](#)
- [gpiod_chip](#), [53](#)
- [gpiod_chip_close](#)
GPIO chips, [8](#)
- [gpiod_chip_get_fd](#)
GPIO chips, [8](#)
- [gpiod_chip_get_info](#)
GPIO chips, [8](#)
- [gpiod_chip_get_line_info](#)
GPIO chips, [8](#)
- [gpiod_chip_get_line_offset_from_name](#)
GPIO chips, [9](#)
- [gpiod_chip_get_path](#)
GPIO chips, [9](#)
- [gpiod_chip_info](#), [53](#)
- [gpiod_chip_info_free](#)
Chip info, [13](#)
- [gpiod_chip_info_get_label](#)
Chip info, [13](#)
- [gpiod_chip_info_get_name](#)
Chip info, [13](#)
- [gpiod_chip_info_get_num_lines](#)
Chip info, [13](#)
- [gpiod_chip_open](#)
GPIO chips, [10](#)
- [gpiod_chip_read_info_event](#)
GPIO chips, [10](#)
- [gpiod_chip_request_lines](#)
GPIO chips, [10](#)
- [gpiod_chip_unwatch_line_info](#)

GPIO chips, [11](#)

- [gpiod_chip_wait_info_event](#)
GPIO chips, [11](#)
- [gpiod_chip_watch_line_info](#)
GPIO chips, [12](#)
- [gpiod_edge_event](#), [54](#)
- [gpiod_edge_event_buffer](#), [54](#)
- [gpiod_edge_event_buffer_free](#)
Line edge events handling, [46](#)
- [gpiod_edge_event_buffer_get_capacity](#)
Line edge events handling, [46](#)
- [gpiod_edge_event_buffer_get_event](#)
Line edge events handling, [47](#)
- [gpiod_edge_event_buffer_get_num_events](#)
Line edge events handling, [47](#)
- [gpiod_edge_event_buffer_new](#)
Line edge events handling, [47](#)
- [gpiod_edge_event_copy](#)
Line edge events handling, [48](#)
- [GPIOD_EDGE_EVENT_FALLING_EDGE](#)
Line edge events handling, [46](#)
- [gpiod_edge_event_free](#)
Line edge events handling, [48](#)
- [gpiod_edge_event_get_event_type](#)
Line edge events handling, [48](#)
- [gpiod_edge_event_get_global_seqno](#)
Line edge events handling, [49](#)
- [gpiod_edge_event_get_line_offset](#)
Line edge events handling, [49](#)
- [gpiod_edge_event_get_line_seqno](#)
Line edge events handling, [49](#)
- [gpiod_edge_event_get_timestamp_ns](#)
Line edge events handling, [50](#)
- [GPIOD_EDGE_EVENT_RISING_EDGE](#)
Line edge events handling, [46](#)
- [gpiod_edge_event_type](#)
Line edge events handling, [46](#)
- [gpiod_info_event](#), [55](#)
- [gpiod_info_event_free](#)
Line status watch events, [23](#)
- [gpiod_info_event_get_event_type](#)
Line status watch events, [23](#)
- [gpiod_info_event_get_line_info](#)
Line status watch events, [23](#)
- [gpiod_info_event_get_timestamp_ns](#)
Line status watch events, [23](#)
- [GPIOD_INFO_EVENT_LINE_CONFIG_CHANGED](#)
Line status watch events, [22](#)
- [GPIOD_INFO_EVENT_LINE_RELEASED](#)

- Line status watch events, [22](#)
- GPIOD_INFO_EVENT_LINE_REQUESTED
 - Line status watch events, [22](#)
- gpiod_info_event_type
 - Line status watch events, [22](#)
- gpiod_is_gpiochip_device
 - Stuff that didn't fit anywhere else, [51](#)
- gpiod_line_bias
 - Line definitions, [14](#)
- GPIOD_LINE_BIAS_AS_IS
 - Line definitions, [15](#)
- GPIOD_LINE_BIAS_DISABLED
 - Line definitions, [15](#)
- GPIOD_LINE_BIAS_PULL_DOWN
 - Line definitions, [15](#)
- GPIOD_LINE_BIAS_PULL_UP
 - Line definitions, [15](#)
- GPIOD_LINE_BIAS_UNKNOWN
 - Line definitions, [15](#)
- gpiod_line_clock
 - Line definitions, [15](#)
- GPIOD_LINE_CLOCK_HTE
 - Line definitions, [15](#)
- GPIOD_LINE_CLOCK_MONOTONIC
 - Line definitions, [15](#)
- GPIOD_LINE_CLOCK_REALTIME
 - Line definitions, [15](#)
- gpiod_line_config, [56](#)
- gpiod_line_config_add_line_settings
 - Line configuration objects, [33](#)
- gpiod_line_config_free
 - Line configuration objects, [34](#)
- gpiod_line_config_get_configured_offsets
 - Line configuration objects, [34](#)
- gpiod_line_config_get_line_settings
 - Line configuration objects, [34](#)
- gpiod_line_config_get_num_configured_offsets
 - Line configuration objects, [35](#)
- gpiod_line_config_new
 - Line configuration objects, [35](#)
- gpiod_line_config_reset
 - Line configuration objects, [35](#)
- gpiod_line_config_set_output_values
 - Line configuration objects, [35](#)
- gpiod_line_direction
 - Line definitions, [15](#)
- GPIOD_LINE_DIRECTION_AS_IS
 - Line definitions, [15](#)
- GPIOD_LINE_DIRECTION_INPUT
 - Line definitions, [15](#)
- GPIOD_LINE_DIRECTION_OUTPUT
 - Line definitions, [15](#)
- gpiod_line_drive
 - Line definitions, [15](#)
- GPIOD_LINE_DRIVE_OPEN_DRAIN
 - Line definitions, [15](#)
- GPIOD_LINE_DRIVE_OPEN_SOURCE
 - Line definitions, [15](#)
- GPIOD_LINE_DRIVE_PUSH_PULL
 - Line definitions, [15](#)
- gpiod_line_edge
 - Line definitions, [15](#)
- GPIOD_LINE_EDGE_BOTH
 - Line definitions, [16](#)
- GPIOD_LINE_EDGE_FALLING
 - Line definitions, [16](#)
- GPIOD_LINE_EDGE_NONE
 - Line definitions, [16](#)
- GPIOD_LINE_EDGE_RISING
 - Line definitions, [16](#)
- gpiod_line_info, [56](#)
- gpiod_line_info_copy
 - Line info, [17](#)
- gpiod_line_info_free
 - Line info, [17](#)
- gpiod_line_info_get_bias
 - Line info, [18](#)
- gpiod_line_info_get_consumer
 - Line info, [18](#)
- gpiod_line_info_get_debounce_period_us
 - Line info, [18](#)
- gpiod_line_info_get_direction
 - Line info, [19](#)
- gpiod_line_info_get_drive
 - Line info, [19](#)
- gpiod_line_info_get_edge_detection
 - Line info, [19](#)
- gpiod_line_info_get_event_clock
 - Line info, [20](#)
- gpiod_line_info_get_name
 - Line info, [20](#)
- gpiod_line_info_get_offset
 - Line info, [20](#)
- gpiod_line_info_is_active_low
 - Line info, [21](#)
- gpiod_line_info_is_debounced
 - Line info, [21](#)
- gpiod_line_info_is_used
 - Line info, [21](#)
- gpiod_line_request, [57](#)
- gpiod_line_request_get_chip_name
 - Line request operations, [39](#)
- gpiod_line_request_get_fd
 - Line request operations, [40](#)
- gpiod_line_request_get_num_requested_lines
 - Line request operations, [40](#)
- gpiod_line_request_get_requested_offsets
 - Line request operations, [40](#)
- gpiod_line_request_get_value
 - Line request operations, [41](#)
- gpiod_line_request_get_values
 - Line request operations, [41](#)
- gpiod_line_request_get_values_subset
 - Line request operations, [41](#)
- gpiod_line_request_read_edge_events
 - Line request operations, [42](#)

- gpiod_line_request_reconfigure_lines
 - Line request operations, [42](#)
- gpiod_line_request_release
 - Line request operations, [43](#)
- gpiod_line_request_set_value
 - Line request operations, [43](#)
- gpiod_line_request_set_values
 - Line request operations, [43](#)
- gpiod_line_request_set_values_subset
 - Line request operations, [44](#)
- gpiod_line_request_wait_edge_events
 - Line request operations, [44](#)
- gpiod_line_settings, [57](#)
- gpiod_line_settings_copy
 - Line settings objects, [25](#)
- gpiod_line_settings_free
 - Line settings objects, [25](#)
- gpiod_line_settings_get_active_low
 - Line settings objects, [26](#)
- gpiod_line_settings_get_bias
 - Line settings objects, [26](#)
- gpiod_line_settings_get_debounce_period_us
 - Line settings objects, [26](#)
- gpiod_line_settings_get_direction
 - Line settings objects, [27](#)
- gpiod_line_settings_get_drive
 - Line settings objects, [27](#)
- gpiod_line_settings_get_edge_detection
 - Line settings objects, [27](#)
- gpiod_line_settings_get_event_clock
 - Line settings objects, [28](#)
- gpiod_line_settings_get_output_value
 - Line settings objects, [28](#)
- gpiod_line_settings_new
 - Line settings objects, [28](#)
- gpiod_line_settings_reset
 - Line settings objects, [28](#)
- gpiod_line_settings_set_active_low
 - Line settings objects, [30](#)
- gpiod_line_settings_set_bias
 - Line settings objects, [30](#)
- gpiod_line_settings_set_debounce_period_us
 - Line settings objects, [30](#)
- gpiod_line_settings_set_direction
 - Line settings objects, [30](#)
- gpiod_line_settings_set_drive
 - Line settings objects, [31](#)
- gpiod_line_settings_set_edge_detection
 - Line settings objects, [31](#)
- gpiod_line_settings_set_event_clock
 - Line settings objects, [32](#)
- gpiod_line_settings_set_output_value
 - Line settings objects, [32](#)
- gpiod_line_value
 - Line definitions, [16](#)
- GPIOD_LINE_VALUE_ACTIVE
 - Line definitions, [16](#)
- GPIOD_LINE_VALUE_ERROR
 - Line definitions, [16](#)
- GPIOD_LINE_VALUE_INACTIVE
 - Line definitions, [16](#)
- gpiod_request_config, [58](#)
- gpiod_request_config_free
 - Request configuration objects, [37](#)
- gpiod_request_config_get_consumer
 - Request configuration objects, [37](#)
- gpiod_request_config_get_event_buffer_size
 - Request configuration objects, [37](#)
- gpiod_request_config_new
 - Request configuration objects, [37](#)
- gpiod_request_config_set_consumer
 - Request configuration objects, [38](#)
- gpiod_request_config_set_event_buffer_size
 - Request configuration objects, [38](#)
- Line configuration objects, [32](#)
 - gpiod_line_config_add_line_settings, [33](#)
 - gpiod_line_config_free, [34](#)
 - gpiod_line_config_get_configured_offsets, [34](#)
 - gpiod_line_config_get_line_settings, [34](#)
 - gpiod_line_config_get_num_configured_offsets, [35](#)
 - gpiod_line_config_new, [35](#)
 - gpiod_line_config_reset, [35](#)
 - gpiod_line_config_set_output_values, [35](#)
- Line definitions, [14](#)
 - gpiod_line_bias, [14](#)
 - GPIOD_LINE_BIAS_AS_IS, [15](#)
 - GPIOD_LINE_BIAS_DISABLED, [15](#)
 - GPIOD_LINE_BIAS_PULL_DOWN, [15](#)
 - GPIOD_LINE_BIAS_PULL_UP, [15](#)
 - GPIOD_LINE_BIAS_UNKNOWN, [15](#)
 - gpiod_line_clock, [15](#)
 - GPIOD_LINE_CLOCK_HTE, [15](#)
 - GPIOD_LINE_CLOCK_MONOTONIC, [15](#)
 - GPIOD_LINE_CLOCK_REALTIME, [15](#)
 - gpiod_line_direction, [15](#)
 - GPIOD_LINE_DIRECTION_AS_IS, [15](#)
 - GPIOD_LINE_DIRECTION_INPUT, [15](#)
 - GPIOD_LINE_DIRECTION_OUTPUT, [15](#)
 - gpiod_line_drive, [15](#)
 - GPIOD_LINE_DRIVE_OPEN_DRAIN, [15](#)
 - GPIOD_LINE_DRIVE_OPEN_SOURCE, [15](#)
 - GPIOD_LINE_DRIVE_PUSH_PULL, [15](#)
 - gpiod_line_edge, [15](#)
 - GPIOD_LINE_EDGE_BOTH, [16](#)
 - GPIOD_LINE_EDGE_FALLING, [16](#)
 - GPIOD_LINE_EDGE_NONE, [16](#)
 - GPIOD_LINE_EDGE_RISING, [16](#)
 - gpiod_line_value, [16](#)
 - GPIOD_LINE_VALUE_ACTIVE, [16](#)
 - GPIOD_LINE_VALUE_ERROR, [16](#)
 - GPIOD_LINE_VALUE_INACTIVE, [16](#)
- Line edge events handling, [45](#)
 - gpiod_edge_event_buffer_free, [46](#)
 - gpiod_edge_event_buffer_get_capacity, [46](#)
 - gpiod_edge_event_buffer_get_event, [47](#)

- gpiod_edge_event_buffer_get_num_events, 47
- gpiod_edge_event_buffer_new, 47
- gpiod_edge_event_copy, 48
- GPIOD_EDGE_EVENT_FALLING_EDGE, 46
- gpiod_edge_event_free, 48
- gpiod_edge_event_get_event_type, 48
- gpiod_edge_event_get_global_seqno, 49
- gpiod_edge_event_get_line_offset, 49
- gpiod_edge_event_get_line_seqno, 49
- gpiod_edge_event_get_timestamp_ns, 50
- GPIOD_EDGE_EVENT_RISING_EDGE, 46
- gpiod_edge_event_type, 46
- Line info, 16
 - gpiod_line_info_copy, 17
 - gpiod_line_info_free, 17
 - gpiod_line_info_get_bias, 18
 - gpiod_line_info_get_consumer, 18
 - gpiod_line_info_get_debounce_period_us, 18
 - gpiod_line_info_get_direction, 19
 - gpiod_line_info_get_drive, 19
 - gpiod_line_info_get_edge_detection, 19
 - gpiod_line_info_get_event_clock, 20
 - gpiod_line_info_get_name, 20
 - gpiod_line_info_get_offset, 20
 - gpiod_line_info_is_active_low, 21
 - gpiod_line_info_is_debounced, 21
 - gpiod_line_info_is_used, 21
- Line request operations, 38
 - gpiod_line_request_get_chip_name, 39
 - gpiod_line_request_get_fd, 40
 - gpiod_line_request_get_num_requested_lines, 40
 - gpiod_line_request_get_requested_offsets, 40
 - gpiod_line_request_get_value, 41
 - gpiod_line_request_get_values, 41
 - gpiod_line_request_get_values_subset, 41
 - gpiod_line_request_read_edge_events, 42
 - gpiod_line_request_reconfigure_lines, 42
 - gpiod_line_request_release, 43
 - gpiod_line_request_set_value, 43
 - gpiod_line_request_set_values, 43
 - gpiod_line_request_set_values_subset, 44
 - gpiod_line_request_wait_edge_events, 44
- Line settings objects, 24
 - gpiod_line_settings_copy, 25
 - gpiod_line_settings_free, 25
 - gpiod_line_settings_get_active_low, 26
 - gpiod_line_settings_get_bias, 26
 - gpiod_line_settings_get_debounce_period_us, 26
 - gpiod_line_settings_get_direction, 27
 - gpiod_line_settings_get_drive, 27
 - gpiod_line_settings_get_edge_detection, 27
 - gpiod_line_settings_get_event_clock, 28
 - gpiod_line_settings_get_output_value, 28
 - gpiod_line_settings_new, 28
 - gpiod_line_settings_reset, 28
 - gpiod_line_settings_set_active_low, 30
 - gpiod_line_settings_set_bias, 30
 - gpiod_line_settings_set_debounce_period_us, 30
 - gpiod_line_settings_set_direction, 30
 - gpiod_line_settings_set_drive, 31
 - gpiod_line_settings_set_edge_detection, 31
 - gpiod_line_settings_set_event_clock, 32
 - gpiod_line_settings_set_output_value, 32
- Line status watch events, 22
 - gpiod_info_event_free, 23
 - gpiod_info_event_get_event_type, 23
 - gpiod_info_event_get_line_info, 23
 - gpiod_info_event_get_timestamp_ns, 23
 - GPIOD_INFO_EVENT_LINE_CONFIG_CHANGED, 22
 - GPIOD_INFO_EVENT_LINE_RELEASED, 22
 - GPIOD_INFO_EVENT_LINE_REQUESTED, 22
 - gpiod_info_event_type, 22
- per_line_config, 59
- Request configuration objects, 36
 - gpiod_request_config_free, 37
 - gpiod_request_config_get_consumer, 37
 - gpiod_request_config_get_event_buffer_size, 37
 - gpiod_request_config_new, 37
 - gpiod_request_config_set_consumer, 38
 - gpiod_request_config_set_event_buffer_size, 38
- settings_node, 59
- src/include/gpiod.h, 61, 66
- src/lib/internal.h, 71
- Stuff that didn't fit anywhere else, 50
 - gpiod_api_version, 51
 - gpiod_is_gpiochip_device, 51